# AI Smart Recipe and Meal Planning Agent

## 1.1 Project Description

**What problem does your project solve?**
Meal planning can be a difficult task for people who have dietary restrictions, specific nutrition goals, limited ingredients, or a limited amount of time to cook. It can also take a significant amount of time to decide what meals to make and figure out what ingredients are needed. My project will help users organize their meals and find recipes based on the information they provide.

**Who are the target users?**
The target users are people who plan and prepare their own meals and want help deciding what to cook. This could include students, working individuals, families, people with busy schedules, or people with dietary restrictions or specific nutrition goals. It could also be useful for users who want to plan meals around ingredients they already have or who are interested in cooking more at home.

**What can the agent do?**
The agent can recommend or generate recipes based on the user's ingredients, available cooking time, dietary restrictions, and preferences. It can also create a weekly meal plan, suggest ingredient substitutions, help use leftover ingredients, and make changes to a meal plan based on the user's requests.

**Why is an AI agent appropriate for this problem?**
Meal planning involves several requirements at once, such as what ingredients a user has, their dietary restrictions, how much time they have to cook, and their nutrition goals. An AI agent can consider these different factors when suggesting recipes and meals that fit the user's needs. The AI chat also allows users to explain what they are looking for in their own words, rather than having to manually select every option. One of the difficult parts of a meal planning app is tabulating and storing a large amount of recipes in a database, but with an AI agent, not only can we leverage the AI to help generate recipes, but we can also store and pass back and forth recipes in plain text.

**Which AI/LLM model(s) do you plan to use?**
We plan to use the Claude Sonnet 5.5 model for the AI portion of the application. The model will be used to understand the user's requests and generate recipe and meal-planning suggestions based on the information provided.

**How will the AI model interact with the rest of the software system?**
The user will enter their requirements through the GUI, such as the ingredients they have, their dietary restrictions, meal preferences, or changes they want to make to their meal plan. When the user asks the AI for a recipe or meal-planning suggestion, this information will be sent to the AI model. The AI will use this information to create a response, which will then be

shown in the GUI, such as in the recipe browser or weekly meal calendar. Other features, like saving recipes, updating the pantry, changing serving sizes, and saving favorites, will be handled by the Java application.

## 1.2 Feature Specification

### F01 - Recipe Recommendation

**Description:**
Recommends recipes based on the user's available ingredients, cooking time, dietary restrictions, and other requirements. Uses recipes recommended from the AI Agent.

**User Interaction:**
The user enters their requirements and description through GUI, and then the app will return recipes from the AI Agent and let users add recipes to their saved recipes. The users can prompt further and ask for more or type out adjustments to their request.

**Input:**
Available ingredients, cooking time, dietary restrictions, meal type, flavour and other preferences

**Output:**
One or more recipe recommendations that match the user's requirements

**AI Involvement:**
AI-based

**Expected Workflow:**
The user enters their requirements → the application sends the information to the AI → the AI recommends suitable recipes → the recommendations are displayed in the GUI → User can ask for more or pick one and/or add it to their saved recipes

**Error/Alternative Cases:**
If there are not enough requirements or no suitable recipe can be found, the system can ask the user for more information or provide alternative suggestions. If the prompt by the user is nonsensical or not related to recipes, the system will instruct the AI agent to flag this and return an error message into the chat.

### F02 - Add Personal Recipe To Recipe List

**Description:**
Allows users to add their own recipes by pasting them in. AI Agent will process it and return the processed/formatted recipe to the system to be added to the saved recipe list.

**User Interaction:**
The user selects the option to add a personal recipe and enters the recipe information through the GUI

**Input:**
Pasted recipe text including recipe name, ingredients, instructions, cooking time, meal type, and other relevant information

**Output:**
The recipe is saved to the user's personal recipe list with its information organized and categorized with fields populated when they exist

**AI Involvement:**
AI-based

**Expected Workflow:**
The user writes/pastes their recipe → the AI analyzes the recipe and parses the information into standardized fields → the system ingests the processed output → the recipe is saved to the personal recipe list

**Error/Alternative Cases:**
If the prompt by the user is not a recipe or nonsensical, the system will ask the AI agent to flag this and return an error and the system will let the user know that the recipe was rejected. If important information is missing, the system will leave those fields blank.

### F03 - View Saved Recipes

**Description:**
Allows users to view their saved recipe collection. The user will be able to view their saved recipes and page through the recipe list. The user will be able to see the recipe details and also remove recipes they do not want anymore.

**User Interaction:**
The user can choose to view their saved recipes in the GUI. The GUI will list recipes by name and allow the user to paginate through them with a set number of recipes per page. The user will be able to select a recipe to view its information and remove any recipes they don’t want.

**Input:**
Selected recipe and user request to remove the recipe.

**Output:**
A list of the user's saved recipes that the user can page through and view. The selected recipe can be removed from the user's saved recipe list.

**AI Involvement:**
Deterministic

**Expected Workflow:**
The user selects to view recipes → The GUI will display the first set number of saved recipes → The user will paginate through the recipes → The user can select a recipe to view its full information → The user can select the option to remove the recipe → The system removes the recipe from the user's saved recipe list → The GUI returns to the updated recipe list

**Error/Alternative Cases:**
If there are no saved recipes, the recipe list will be empty. If the user tries to remove a recipe that no longer exists in their saved recipe list, the system will display an error message and refresh the recipe list.

**F04 - Edit Saved Recipe**

**Description:**
Allows users to edit the information of a saved recipe. The user will be able to select a saved recipe from the recipe list and edit the recipe fields such as the recipe name, ingredients, instructions, cooking time, meal type, and other relevant information.

**User Interaction:**
When a user selects a recipe in the recipe list, they will have the option to edit the recipe. The user selects edit and then is able to edit the selected recipe's information. The user is then able to select save to save their changes, or select cancel to exit edit mode and discard any changes that were made.

**Input:**
Updated recipe name, ingredients, instructions, cooking time, meal type, and other relevant recipe information.

**Output:**
The saved recipe is updated with the user's changes and the updated information is displayed when viewing the recipe.

**AI Involvement:**
Deterministic

**Expected Workflow:**
The user selects to view recipes → The GUI will display the saved recipes → The user selects a recipe → The user selects the option to edit the recipe → The GUI will display the recipe fields with their current information → The user edits one or more recipe fields → The user selects to save the changes → The system updates the saved recipe with the new information → The GUI displays the updated recipe.

**Error/Alternative Cases:**
If the user attempts to save the recipe with required information missing, the system will let the fields be blank. If the user changes the name of the recipe to one that already exists, the system will block saving.

## F05 - Recipe Modification Help

**Description:**
Users can select a saved/added recipe and ask for alternative ingredients, serving size adjustments, or cooking modifications. The system and AI agent will give suggestions and allow the option of saving the modified recipe

**User Interaction:**
Users select a recipe in the recipe list in the GUI and can choose to ask for a modification. The user will then be able to describe the modification they want. The AI agent will update the recipe in the edit GUI. The user can choose to save the modified recipe by selecting save in the GUI which will take it out of edit mode.

**Input:**
User description of what modifications they want

**Output:**
AI recommendation of how to modify the recipe of what ingredients might be a good substitute

**AI Involvement:**
AI-based

**Expected Workflow:**
User selects a recipe in the GUI → User selects option to ask for modification → User describes the modification they want in the AI chat → AI agent will reply with suggestions and advice → User can continue to talk and discuss with AI agent or tell the AI agent to modify the recipe in edit mode → the user then can choose to select save or cancel in the GUI whenever they want.

**Error/Alternative Cases:**
If the prompt by the user is not recipe related or nonsensical, the system will ask the AI agent to flag this and return an error text as a reply.

**F06 - Search Saved Recipes**

**Description:**
Allows users to search through their saved recipe collection and find recipes by describing the criteria they want. AI Agent will search through the user's saved recipe collection and return recipes that match the user's prompt.

**User Interaction:**
When viewing their saved recipes in the GUI, the user can select the option to search/filter recipes and type a search and filter description. The system will then send over the user's saved recipes along with their search description to the AI Agent, which will return a list of matching recipes. The system will filter down the recipe list to these matching recipes and the user will be able to page through them.

**Input:**
User description of search terms, ingredients, cooking time, meal type, dietary restrictions, or other filters.

**Output:**
A list of recipes that match the user's described criteria.

**AI Involvement:**
AI-based

**Expected Workflow:**
The user selects to view recipes → The GUI will display the saved recipes → The user selects the option to search/filter recipes → The user describes their search criteria → The system sends the saved recipes and search description to the AI Agent → The AI Agent returns a list of matching recipes → The GUI will filter down the list of recipes to those that the AI Agent matches to the search description → The user will paginate through the recipes.

**Error/Alternative Cases:**
If the prompt by the user is not recipe related or nonsensical, the system will ask the AI agent to flag this and return an error text as a reply. If no recipes match the search, the recipe list will be empty. If there are no saved recipes, the recipe list will be empty.

**F07 - Weekly Meal Planning**

**Description:**
Allows the user to meal-plan on a weekly calendar. The user will be able to add saved recipes or newly recommended recipes to meal time slots on the calendar. The user can also have the AI agent generate the meal plan.

**User Interaction:**
The user can see a layout of the week and meal times on the GUI. The user can select a day and a meal time on the GUI and then browse and search through the list of recipes they have saved using the F03 feature and add it to that slot on the weekly calendar. They can also select the option to ask the AI agent to generate a meal plan based on a written description of criteria like nutrition goals, preferences, dietary restrictions, available ingredients, cooking utensils, current pantry etc. The user will be able to ask the AI agent to generate a meal plan using saved recipes or recommended recipes and see all this populated on the weekly calendar.

**Input:**
Selected recipes, the day, and meal timeslots. Nutrition goals, dietary restrictions, food preferences, available ingredients, preferred meal types and any other criteria. Whether or not the user wants to use only saved recipes.

**Output:**
A manually created or AI suggested weekly meal plan displayed on the meal calendar.

**AI Involvement:**
Hybrid

**Expected Workflow:**
The user selects the weekly meal planner → The GUI displays the weekly calendar with days and meal times → The user selects a day and meal timeslot → The user selects the option to add a recipe → The user browses or searches their saved recipes using F03 → The GUI displays matching saved recipes → The user selects a recipe → The updated meal plan is displayed on the weekly calendar.

The user selects the weekly meal planner → The user selects the option to generate a meal plan using AI → The user describes their meal-planning criteria, such as nutrition goals, dietary restrictions, food preferences, available ingredients, pantry contents, cooking equipment, and preferred meal types → The system sends the user's criteria to the AI Agent → The AI Agent generates a weekly meal plan using suitable saved recipes → The system receives and processes the generated meal plan → The generated recipes are assigned to the appropriate days and meal time slots → The completed meal plan is displayed on the weekly calendar

**Error/Alternative Cases:**
If the user has no saved recipes when attempting to use saved recipes, the system will display an empty list. If the AI Agent cannot generate a suitable meal plan based on the provided criteria, the system will ask the user for additional information or suggest relaxing some requirements. If no suitable recipes can be found for a particular meal slot, or if there are no saved recipes at all, the system will leave the slots empty. If the user's prompt is nonsensical or unrelated to meal planning, the system will ask the AI Agent to

flag this and return an error message. The user can also manually replace or remove recipes that were added to the meal plan via the Agent.

**F08 - Shopping List Generation**

**Description:**
Creates a shopping list based on the recipes in the user's weekly meal plan and the ingredients they already have in their pantry.

**User Interaction:**
On the meal planner GUI they can select to generate a shopping list based on the selected recipes for the list. The User will be able to select whether or not to factor in their current pantry when generating the meal plan. The AI Agent will generate a shopping list and display it to the user.

**Input:**
Recipes in the meal plan and ingredients currently in the user's pantry

**Output:**
A shopping list of ingredients that need to be purchased

**AI Involvement:**
AI-based

**Expected Workflow:**
The user selects the option to generate a shopping list from the meal planner GUI → The user selects whether to factor in their current pantry → If the user chooses to factor in their pantry, the system retrieves the user's current pantry ingredients → The system sends the selected recipes and pantry information to the AI Agent → The AI Agent determines the ingredients and quantities required for the selected recipes → The AI Agent generates a shopping list containing the ingredients and quantities that still need to be purchased → The system displays the generated shopping list in the GUI.

**Error/Alternative Cases:**
If the user already has all the required ingredients, the system can show that no additional items are needed. If a recipe is missing ingredient information, the system can notify the user

**F09 - Favourite Recipes**

**Description:**
Allows users to mark saved recipes as favourite when viewing them through the recipe list. Favourited recipes will be tracked and this information will be passed to the AI agent

for consideration for all AI features.

**User Interaction:**
When viewing recipes in the recipe list, there will be an indicator if a recipe is a favourite. The user can open the recipe details and will have the option to favourite or unfavourite the recipe.

**Input:**
The selected recipe and the user's choice to favourite or unfavourite the recipe.

**Output:**
The recipe is saved to or removed from the user's favourites. The favourite status will be tracked and passed to the AI agent when other AI features are used.

**AI Involvement:**
Deterministic

**Expected Workflow:**
The user selects to view a recipe → The GUI displays the recipe details and whether the recipe is currently a favourite → The user selects the option to favourite or unfavourite the recipe → The system saves update → The GUI updates the recipe to show that it is a favourite.

**Error/Alternative Cases:**
The system will only let you favourite saved recipes. This feature will not be supported for non-saved recommended recipes.

### F10 - Pantry Management

**Description:**
Allows users to keep track of the ingredients they currently have available at home. This information will be passed to the AI agent for AI related features when relevant.

**User Interaction:**
The user can open the pantry section of the GUI to view, add, remove, or edit pantry items. Pantry items will have a name, and a quantity. The GUI will show the list of current pantry items and the user will be able to add new items and then enter a name and a quantity. They will also be able to select to remove pantry items and edit both the name and the quantity for existing items.

**Input:**
Ingredient name and quantity

**Output:**
An updated list of ingredients and their quantities in the user's pantry

**AI Involvement:**
Deterministic

**Expected Workflow:**
The user selects to view the pantry → The GUI displays the current list of pantry items with their names and quantities → The user can select to add a new pantry item → The user enters the ingredient name and quantity → The system saves the new pantry item → The GUI updates the pantry list to include the new item.

The user selects an existing pantry item → The user selects the option to edit the pantry item → The user edits the ingredient name and/or quantity → The system saves the updated pantry item → The GUI updates the pantry list with the new information.

The user selects an existing pantry item → The user selects the option to remove the pantry item → The system removes the pantry item from the user's pantry → The GUI updates the pantry list.

**Error/Alternative Cases:**
If the user adds something that is not a pantry item, or if the user enters in something nonsensical, the system will leave it be. The quality of the AI features relating to the pantry functionality depend on the quality of the user's entries in the pantry list. There will be a character limit for the pantry item names. If the user attempts to add a pantry item without an ingredient name or quantity, the system will ask the user to provide the missing information before saving. If the user attempts to edit a pantry item with invalid or missing information, the system will ask the user to correct the information before saving. If the user tries to edit or add a name that already exists in the pantry, the system will return an error message. If there are no pantry items, the pantry list will be empty.

**F11 - User Preference Profiles and Dietary Restrictions**

**Description:**
Allows users to save their general food preferences and dietary restrictions so they can be used and passed to the AI agent when recommending recipes and creating meal plans

**User Interaction:**
The user can select to view the profile in the GUI. The GUI will show fields for their dietary restrictions, preferred foods, and disliked ingredients. There will also be a field for general nutritional goals. The user can select any of these fields to edit in the GUI.

**Input:**
Dietary restrictions, food preferences, disliked ingredients, nutrition goals, and other relevant preferences

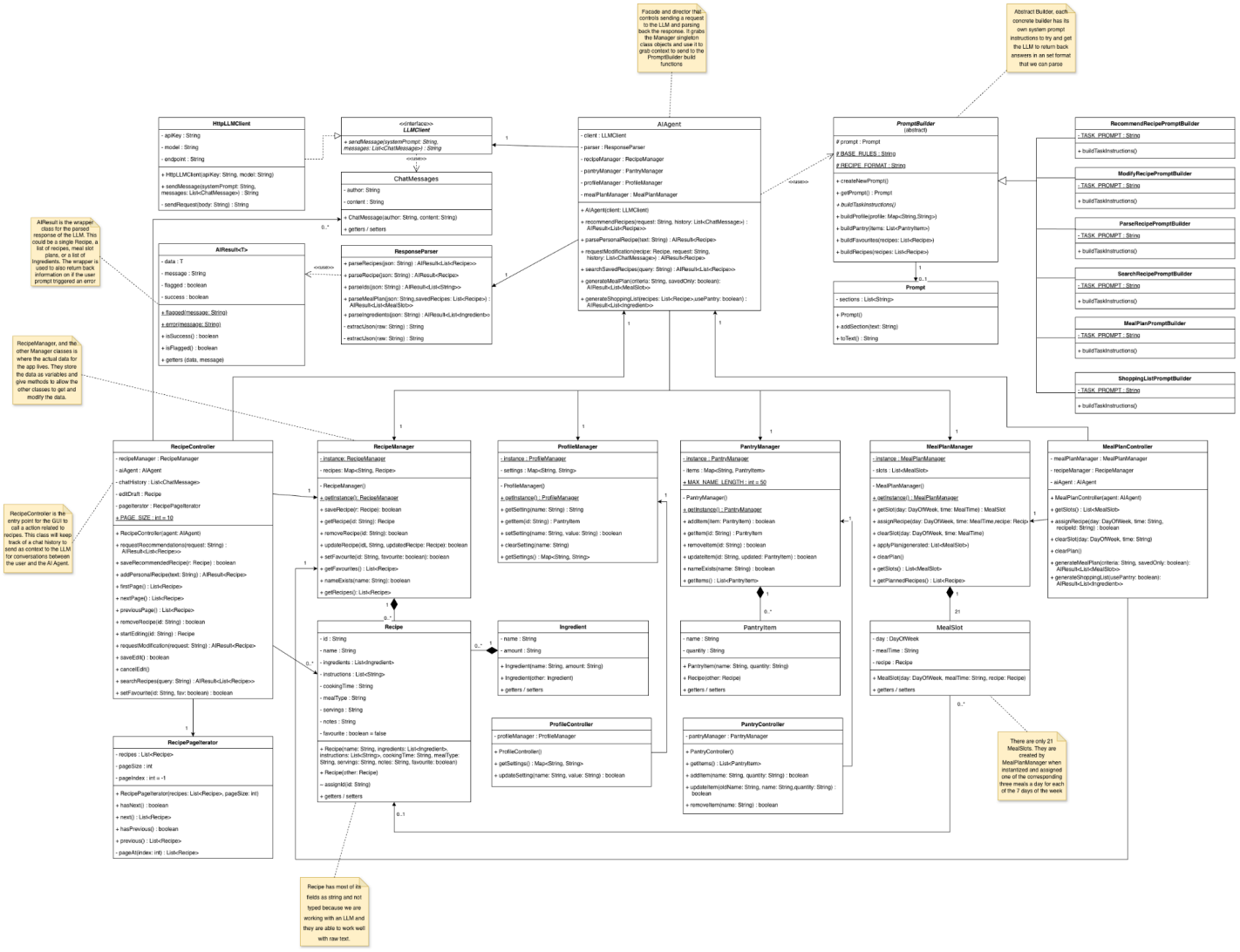
**Output:**
The user's preferences are saved to their profile and will be passed to the AI agent when other features are used

**AI Involvement:**
Deterministic

**Expected Workflow:**
The user selects to view their profile → The GUI displays their saved dietary restrictions, food preferences, disliked ingredients, nutrition goals, and other relevant preferences → The user selects a field they want to edit → The user enters or updates their preferences → The user saves their changes → The updated preferences are displayed in the GUI.

**Error/Alternative Cases:**
If the user leaves a preference field blank, the system will save the field as blank and the AI agent will not use that preference when recommending recipes or creating meal plans. If the user writes nonsensical information in the fields, the system will add a hidden prompt when using AI features to flag it in the response. The system will parse this flag and notify the user in the GUI and wipe the field from the profile.

## 2.1 Class Diagram




[Click here to view the full-size class diagram]

**Design Patterns**

**Singleton**

- We only want one exact copy of manager classes. These classes were designed to manage all of a one data store for the app and are not actually intended to be instantiated more than once. Furthermore, these manager classes are the global data store of recipes, profile settings, pantry list, and meal plan and we need a global instance of them for all classes to use. The problem is then how do we design these global state managers that should only have one instance that needs to be shared by all classes that use them.
- RecipeManager, PantryManager, ProfileManager, MealPlanManager, RecipeController, MealPlanController, PantryController, ProfileController, AIAgent
- RecipeManager, PantryManager, ProfileManager, and MealPlanManager and each individual implementations of the Singleton Design Pattern. RecipeController, MealPlanController, PantryController, ProfileController, and AIAgent are users of those Singleton classes. They each grab the shared global instance of the manager classes they need when they try to create a new object.
- This pattern is appropriate because a Singleton should be used when there should only be one object of its type and it needs to provide a global access point to that instance. This is the case here for RecipeManager, PantryManager, ProfileManager, MealPlanManager. Each of the controllers that run the actual features needs access to the same shared object as the AIAgent that needs it to append to the prompt.
- Nothing would stop a second manager being created. A controller that made its own RecipeManager would hold a separate list, so recipes saved on one screen would be missing on another, and duplicate-name checks would miss names stored in the other copy. Also these shared instances would need to be passed through in the constructors and function calls for everything that needed them which is a lot messier than allowing the users to grab it themselves through the global access point.

**Builder**

- Each of the six AI features needs a system prompt with the same structure: base rules and instructions to respect certain behaviors, task instructions, then optional context sections for the profile, pantry, favourites and recipes. However, each feature's system prompt differs slightly. The structure is fixed and we want to create a prompt for each feature but the task text and which context is included differ per feature.
- PromptBuilder, RecommendedRecipePromptBuilder, ParseRecipePromptBuilder, ModifyRecipePromptBuilder, SearchRecipePromptBuilder, MealPlanPromptBuilder, ShoppingListPromptBuilder, AIAgent, Prompt
- PromptBuilder is the abstract class that contains the build steps and share functions that each specific builder inherits. RecommendedRecipePromptBuilder, ParseRecipePromptBuilder, ModifyRecipePromptBuilder, SearchRecipePromptBuilder, MealPlanPromptBuilder, ShoppingListPromptBuilder are the six concrete builders that each have their own task prompt by implementing

their specific buildTaskInstructions function. AIAgent is the director that will be using the concrete builders.

- This is appropriate because we want to have one construction process that can produce different representations of the prompt without having to replicate code paths. The abstract builder has all the shared code and each concrete builder adds only what they do differently for the task prompt.
- We would need to have 6 duplicate code paths, one for each different prompt. We would need to have code to construct the base rules system prompt, add all the recipes' context. Then this code would be duplicated but changes slightly six times and if we needed to edit a shared function, we would need to remember to change it in all six places.

**Facade**

- Every AI request needs several classes working together in to do similar things like pick the right prompt builder, read context from the managers, build the prompt, send it through the LLM client, parse the reply, and wrap the outcome. The controllers only want to do things like "recommend recipes" or "generate a shopping list" and get a result back. The problem is how we give the controllers a simple way to use a complicated AI subsystem without them needing to know about all of its classes.
- AIAgent, PromptBuilder and its six concrete builders, Prompt, LLMClient, HttpLLMClient, ResponseParser, AIResult, RecipeController, MealPlanController
- AIAgent is the facade, it has one method per AI feature, and each method runs the build, send, and parse sequence and returns an AIResult. The PromptBuilder classes, Prompt, LLMClient, HttpLLMClient, and ResponseParser are the classes that do the actual work and are never directly used by the controller classes. AIResult is the single result type the facade hands back. RecipeController and MealPlanController are the clients that call AIAgent's methods and decide what to do with the result, such as saving a recipe or applying a meal plan.
- This pattern is appropriate because a Facade should be used when we want to provide a simple interface to a complex subsystem so that the application doesn’t have to contain every complex function call for AI features that follow the same sequence. Putting it behind one class keeps it consistent and keeps the controllers simple.
- Each controller would need to know about the prompt builders, the HTTP client, JSON parsing, and the nonsense flag format, and would repeat the same sequence for every feature. Any change to the parser or the prompt format would spread into the controllers instead of staying inside the AI classes.

**Iterator**

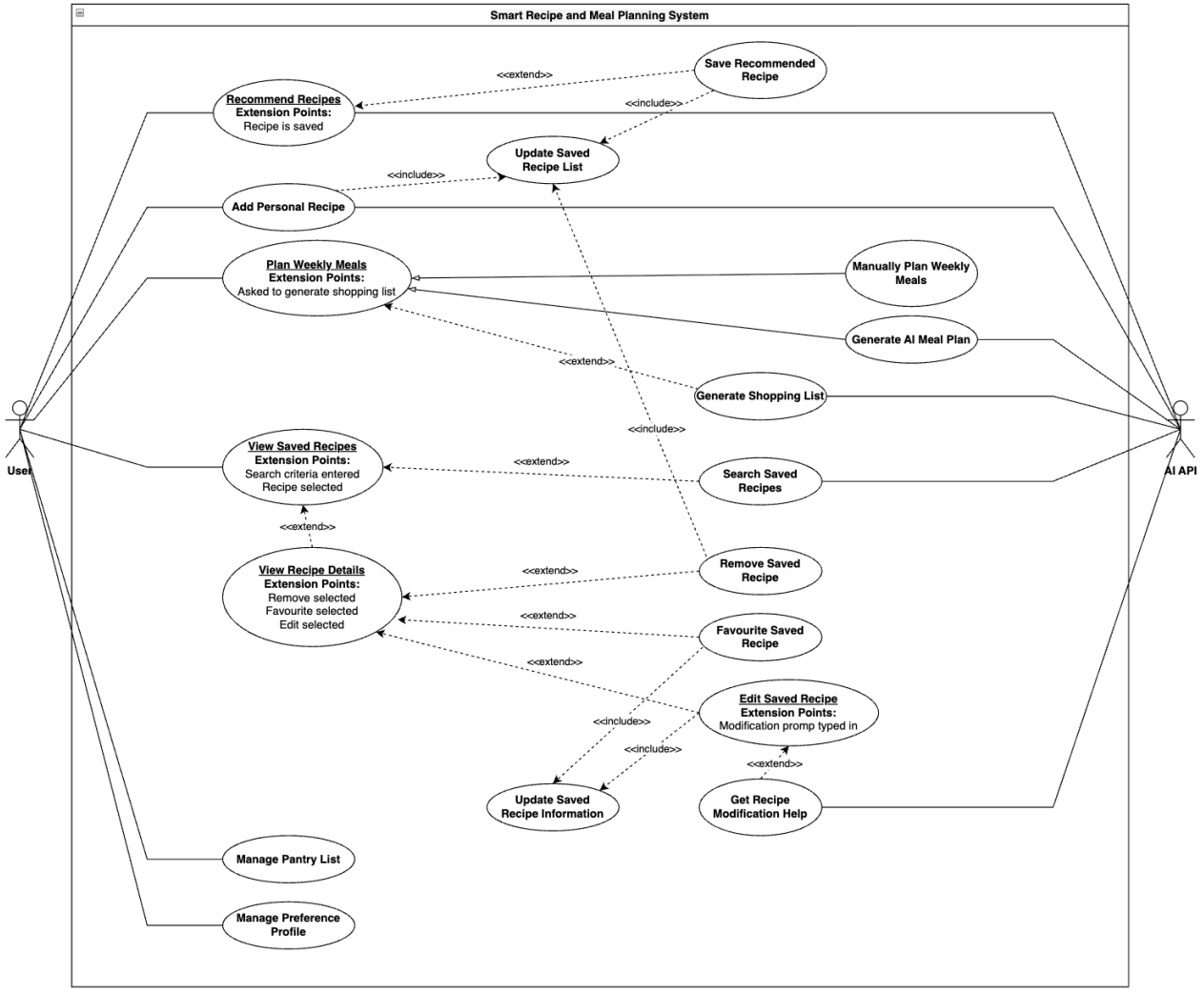
- The saved recipe list shows a set number of recipes per page, and the user can go forward and back between pages (F03). Search results need to be handled the same way (F06). The problem is we want to give the application and the user a way to look through a list of recipes in set sections and keep a cursor of where the user is currently at.
- RecipePageIterator, RecipeController, RecipeManager

- RecipePageIterator is the iterator and it holds a copy of the recipes list returned by the RecipeManager, the page size, and the current page, and has the functions hasNext(), next(), hasPrevious(), and previous(), each returning one page of recipes. The list of recipes it walks through is the collection, either all saved recipes from RecipeManager.getRecipes() or the matches from a search. RecipeController is the client: it creates a new iterator in firstPage() and searchRecipes(), moves with nextPage() and previousPage(), and uses hasNextPage() and hasPreviousPage() so the GUI knows which arrow buttons to enable.
- This pattern is appropriate because an Iterator should be used when we want to traverse a collection without the client knowing how the traversal works. The same iterator works for both the full list and search results. Because it works on a list of recipe instances, removing a recipe while browsing can't break it and the controller would just starts a new one.
- RecipeController would have to track page numbers and slice the lists by hand, separately for saved recipes and search results. The checks for the first and last page would be mixed in with the chat, editing, and favourite code in the same class.

**Adapter**

- The AI features need to talk to an external LLM service through its web API, which has its own request format, authentication, timeouts, and errors that don't match how our app wants to call an AI. We want the rest of the app to just send a prompt and the chat history and get text back. The problem is how we let our classes use an outside service whose interface we don't control and that doesn't match the interface our app expects.
- LLMClient, HttpLLMClient, the vendor's LLM web API, AIAgent, ChatMessage
- LLMClient is the target interface that the app expects, with one method, sendMessage(systemPrompt, messages). HttpLLMClient is the adapter and it implements LLMClient and translates each sendMessage call into the vendor's HTTP request, handling the API key, endpoint, timeout, and retries, then returns the reply text. The vendor's LLM web API is the thing being adapted, the external service with the incompatible interface. AIAgent is the client that only ever calls LLMClient. ChatMessage is the data passed through the interface.
- This pattern is appropriate because an Adapter should be used when we need to use an existing service whose interface doesn't match the one that makes sense for the app. We don't control the vendor's API, so one class absorbs the mismatch. Since AIAgent only depends on the LLMClient interface, we can switch to a different LLM by writing another adapter without changing anything else.
- All the LLM model specific request building, headers, JSON formats, and error handling would live inside AIAgent. Switching models or vendors would mean rewriting AIAgent, and testing would need real, paid API calls, because there would be no interface to substitute a fake client for.

## 2.2 Use-Case Diagram and Use-Case Descriptions




[Click here to view the full-size class diagram]

**Actors:** The primary actor is the User. The system also interacts with an external AI API for AI-based functionality. No administrator actor is included because the current system does not have administrator functionality.

**UC01 - Recommended Recipes**

- **Use Case ID:** UC01
- **Use Case Name:** Recommended Recipes
- **Actor(s):** User, AI API
- **Goal:** Allow the user to receive recipe recommendations based on their available ingredients, cooking time, dietary restrictions, meal type, and other preferences
- **Preconditions:** The user is in the app and can access the recipe recommendation feature. The AI API is available.
- **Trigger:** The user enters their requirements and requests recipe recommendations
- **Main Success Scenario:**
    - The user enters their recipe requirements (ingredients, cooking time, dietary restrictions, meal type, flavour and other preferences) through the GUI
    - The system sends the requirements to the AI API
    - The AI API generates suitable recipe recommendations
    - The system receives the recommendations
    - The system displays the recommended recipes to the user in the GUI
    - The user reviews the recipes and can ask for additional recommendations
    - The user can select a recommended recipe to save
- **Alternative/Exception Flows**
    - If there is not enough information the system asks the user for additional requirements
    - If no suitable recipe can be found, the system provides alternative suggestions
    - If the user's request is nonsensical or unrelated to recipes, the system displays an error
- **Postconditions:** Recipe recommendations are displayed to the user, and the user may save a selected recommendation
- **Related Feature(s):** F01 – Recipe Recommendation

**UC02 - Add Personal Recipe**

- **Use Case ID:** UC02
- **Use Case Name:** Add Personal Recipe
- **Actor(s):** User, AI API
- **Goal:** Allow the user to add their own recipe to their saved recipe collection
- **Preconditions:** The user is in the app and has a recipe they want to add. The AI API is available.
- **Trigger:** The user selects the option to add a personal recipe
- **Main Success Scenario:**
    - The user enters or pastes their recipe into the GUI
    - The system sends the recipe information to the AI API

    - The AI API processes and organizes it into standardized fields (name, ingredients, instructions, time, meal type, etc.)
    - The system receives the processed recipe
    - The system adds the recipe to the user's saved recipe list
    - The system confirms the recipe was added
- **Alternative/Exception Flows**
    - If the input is not a recipe or is nonsensical, the system rejects the recipe and informs the user
    - If some recipe information is missing, the system leaves those fields blank
- **Postconditions:** The personal recipe is added to the user's saved recipe collection
- **Related Feature(s):** F02 – Add Personal Recipe

**UC03 - View Saved Recipes**

- **Use Case ID:** UC03
- **Use Case Name:** View Saved Recipes
- **Actor(s):** User
- **Goal:** Allow the user to view and manage their saved recipe collection
- **Preconditions:** The user is in the app and can access their saved recipe collection
- **Trigger:** The user selects the saved recipes section
- **Main Success Scenario:**
    - The system displays the first page of saved recipes by name
    - The user can page through the recipe list
    - The user selects a recipe
    - The system displays the recipe details
    - The user can choose to remove the recipe
- **Alternative/Exception Flows**
    - If there are no saved recipes, the system displays an empty recipe list
    - If a selected recipe no longer exists, the system displays an error and refreshes the recipe list
- **Postconditions:** The user has viewed their saved recipes, and any requested removal is reflected in the updated recipe list
- **Related Feature(s):** F03 – View Saved Recipes

**UC04 - Edit Saved Recipe**

- **Use Case ID:** UC04
- **Use Case Name:** Edit Saved Recipe
- **Actor(s):** User
- **Goal:** Allow the user to modify the information of a saved recipe
- **Preconditions:** The selected recipe exists in the user's saved recipe collection
- **Trigger:** The user selects a saved recipe and chooses the edit option
- **Main Success Scenario:**
    - The user selects a saved recipe
    - The system displays the recipe's current information
    - The user selects the edit option

    - The user changes one or more recipe fields
    - The user selects the save option
    - The system updates the saved recipe information
    - The GUI displays the updated recipe
- **Alternative/Exception Flows**
    - The user can cancel editing and discard their changes
    - If the user changes the recipe name to one that already exists, the system prevents the recipe from being saved
    - Optional fields can remain blank
- **Postconditions:** The saved recipe reflects the edits, or is unchanged if cancelled
- **Related Feature(s):** F04 – Edit Saved Recipe

**UC05 - Get Recipe Modification Help**

- **Use Case ID:** UC05
- **Use Case Name:** Get Recipe Modification Help
- **Actor(s):** User, AI API
- **Goal:** Allow the user to ask the AI for suggestions for modifying a saved recipe (including substitutions, serving size changes or cooking changes)
- **Preconditions:** The selected recipe exists in the user's saved recipes. The AI API is available.
- **Trigger:** The user selects a recipe and requests modification help
- **Main Success Scenario:**
    - The user selects a saved recipe
    - The user selects the option to get modification help
    - The user describes the modification they want in the AI chat
    - The system sends the recipe and the request to the AI API
    - The AI API provides suggestions or modifications
    - The system displays the AI's response to the user
    - The user can continue discussing the modification or apply the suggested changes
    - The user can save or cancel the modified recipe
- **Alternative/Exception Flows**
    - If the request is nonsensical or unrelated to recipe modification, the system displays an error response
- **Postconditions:** The recipe is changed only if the user saves
- **Related Feature(s):** F05 – Recipe Modification Help

**UC06 - Search Saved Recipes**

- **Use Case ID:** UC06
- **Use Case Name:** Search Saved Recipes
- **Actor(s):** User, AI API
- **Goal:** Allow the user to find saved recipes based on criteria they describe
- **Preconditions:** The user can access their saved recipe collection. The AI API is available.

- **Trigger:** The user enters search or filtering criteria while viewing the saved recipes list
- **Main Success Scenario:**
    - The user opens their saved recipes
    - The user selects the search/filter option
    - The user describes the criteria they want
    - The system sends the saved recipes and search criteria to the AI API
    - The AI API identifies matching recipes
    - The system displays the matching recipes
    - The user can page through the search results
- **Alternative/Exception Flows**
    - If there are no matching recipes, the system displays an empty result list
    - If there are no saved recipes, the result list is empty
    - If the search request is nonsensical or unrelated to recipes, the system displays an error
- **Postconditions:** The user is shown the saved recipes that match their search criteria
- **Related Feature(s):** F06 – Search Saved Recipes

**UC07 - Plan Weekly Meals**

- **Use Case ID:** UC07
- **Use Case Name:** Plan Weekly Meals
- **Actor(s):** User, AI API
- **Goal:** Allow the user to create and manage a weekly meal plan
- **Preconditions:** The user can access the weekly meal planner
- **Trigger:** The user opens the weekly meal planner
- **Main Success Scenario:**
    - The system displays the weekly meal calendar
    - The user selects a day and meal time
    - The user chooses a recipe to add to the selected time slot
    - The system adds the recipe to the meal plan
    - The system displays the updated meal plan
- **Alternative/Exception Flows**
    - The user selects to use AI to generate a weekly meal plan
        - The user selects to generate a weekly meal plan
        - The user selects whether or not to use profile preferences
        - The user selects whether or not to use only recipe whose ingredients already exist in the pantry
        - AI Agent generates the meal plan
        - The system displays the updated meal plan
    - If there are no saved recipes, the recipe list is empty
    - If the AI cannot generate a suitable meal plan, the system asks the user for additional information or suggests relaxing some requirements
    - If no suitable recipe exists for a meal slot, the slot remains empty
    - The user can manually replace or remove recipes generated by the AI

- **Postconditions:** A weekly meal plan is displayed containing manually selected and/or AI-generated meals
- **Related Feature(s):** F07 – Weekly Meal Planning

**UC08 - Generate Shopping List**

- **Use Case ID:** UC08
- **Use Case Name:** Generate Shopping List
- **Actor(s):** User, AI API
- **Goal:** Generate a shopping list based on recipes in the user's meal plan and ingredients already available in their pantry
- **Preconditions:** A meal plan with at least one recipe exists. The AI API is available.
- **Trigger:** The user selects the option to generate a shopping list
- **Main Success Scenario:**
    - The user selects the shopping list option from the meal planner
    - The user chooses whether to consider their pantry
    - If requested, the system retrieves the user's pantry information
    - The system sends the selected recipes and pantry information to the AI API
    - The AI API determines which ingredients are still needed
    - The AI API generates a shopping list
    - The system displays the shopping list to the user
- **Alternative/Exception Flows**
    - If all required ingredients are already available, the system indicates that no additional items are needed
    - If a recipe is missing ingredient information, the system notifies the user
- **Postconditions:** A shopping list containing the ingredients that need to be purchased is displayed
- **Related Feature(s):** F08 – Shopping List Generation

**UC09 - Favourite Saved Recipe**

- **Use Case ID:** UC09
- **Use Case Name:** Favourite Saved Recipe
- **Actor(s):** User
- **Goal:** Allow the user to mark saved recipes as favourites or remove their favourite status
- **Preconditions:** The selected recipe is saved in the user's recipe collection
- **Trigger:** The user selects the favourite or unfavourite option while viewing a saved recipe
- **Main Success Scenario:**
    - The user views a saved recipe
    - The system displays the recipe's current favourite status
    - The user chooses to favourite or unfavourite the recipe
    - The system updates the favourite status
    - The GUI displays the updated status
- **Alternative/Exception Flows**

    - The user cannot favourite a recipe that has not been saved
- **Postconditions:** The recipe's favourite status is updated
- **Related Feature(s):** F09 – Favourite Recipes

**UC10 - Manage Pantry List**

- **Use Case ID:** UC10
- **Use Case Name:** Manage Pantry List
- **Actor(s):** User
- **Goal:** Allow the user to keep track of ingredients currently available at home
- **Preconditions:** The user can access the pantry section
- **Trigger:** The user opens the pantry section
- **Main Success Scenario:**
    - The system displays the user's current pantry items with names and quantities
    - The user can select the option to add a pantry item
    - The user enters the ingredient name and quantity
    - The system saves the pantry item
    - The system updates the pantry list
- **Alternative/Exception Flows**
    - The user can also select to remove a pantry item
    - The user can also select to edit a pantry item, and edit only its quantity
- **Postconditions:** The pantry contains the user's updated list of ingredients and quantities
- **Related Feature(s):** F10 – Pantry Management

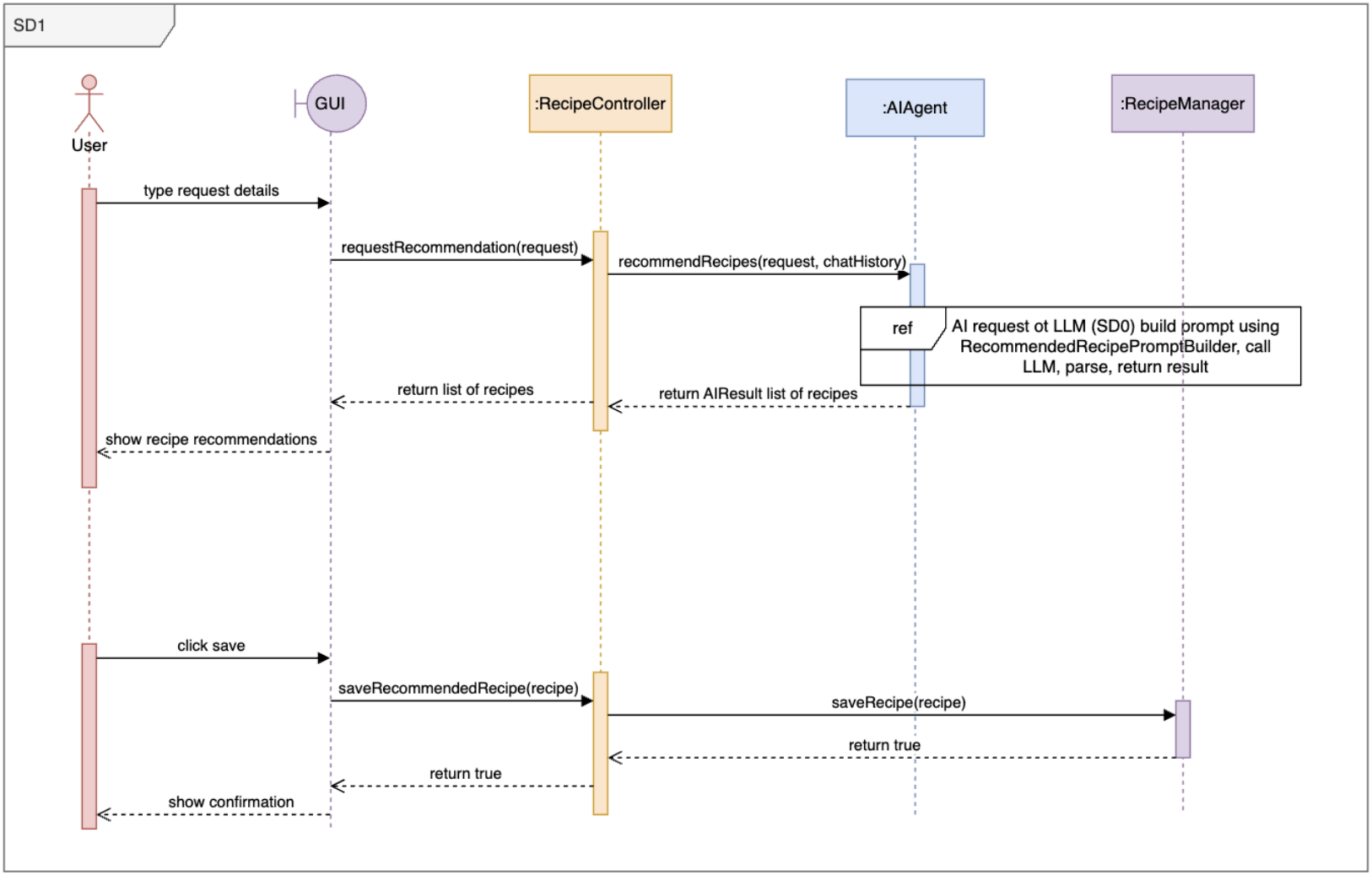
**UC11 - Manage Preference Profile**

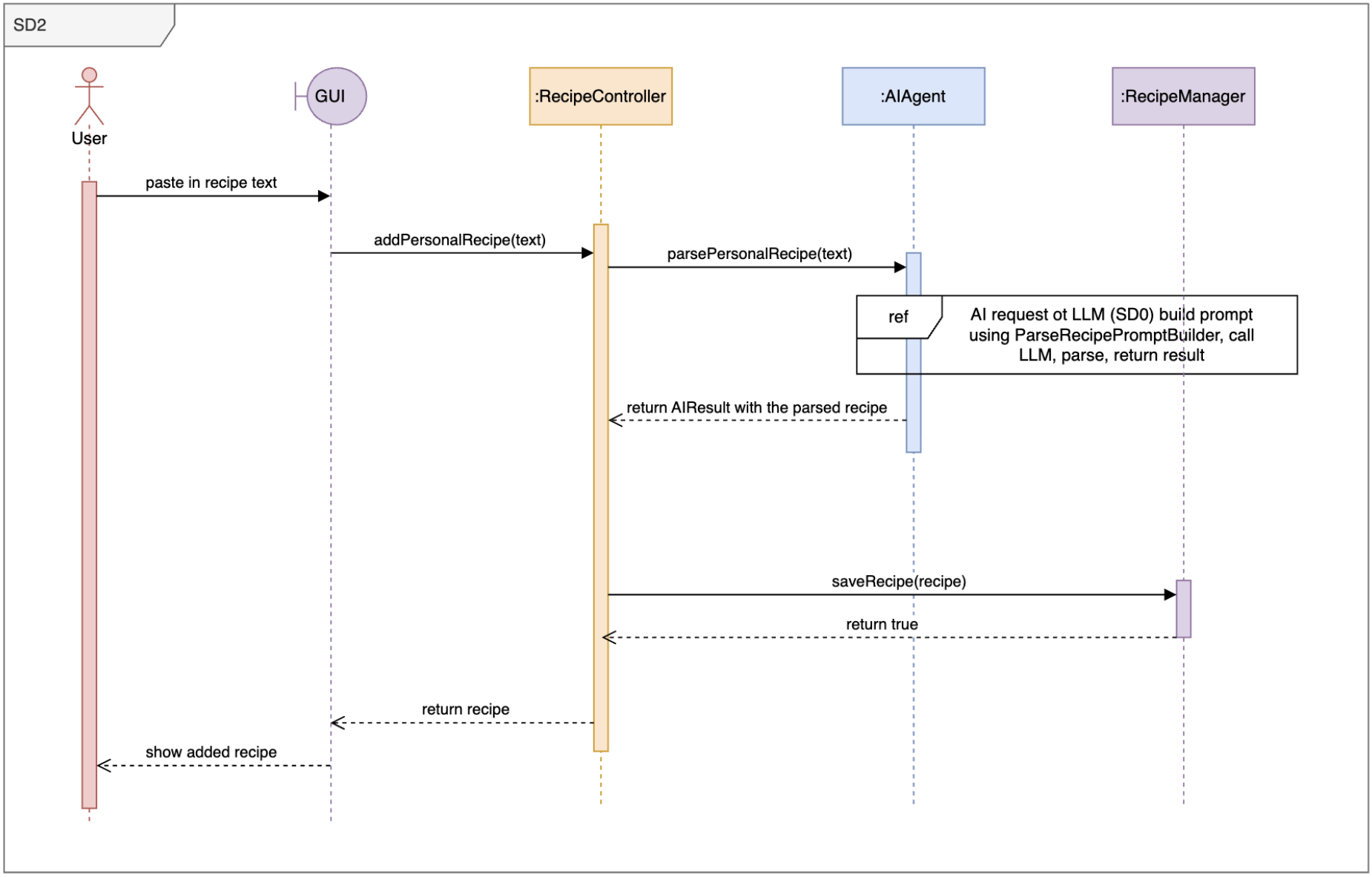
- **Use Case ID:** UC11
- **Use Case Name:** Manage Preference Profile
- **Actor(s):** User
- **Goal:** Allow the user to save and update their food preferences and dietary restrictions for AI features to use.
- **Preconditions:** The user can access their preference profile
- **Trigger:** The user opens their preference profile
- **Main Success Scenario:**
    - The system displays the user's saved preferences
    - The user selects a preference field to edit
    - The user enters or updates their information
    - The user saves the changes
    - The system updates the preference profile
    - The updated preferences are displayed
- **Alternative/Exception Flows**
    - If a preference is left blank, the system saves the field as blank
    - If invalid or nonsensical information is entered, the system notifies the user and removes the invalid information

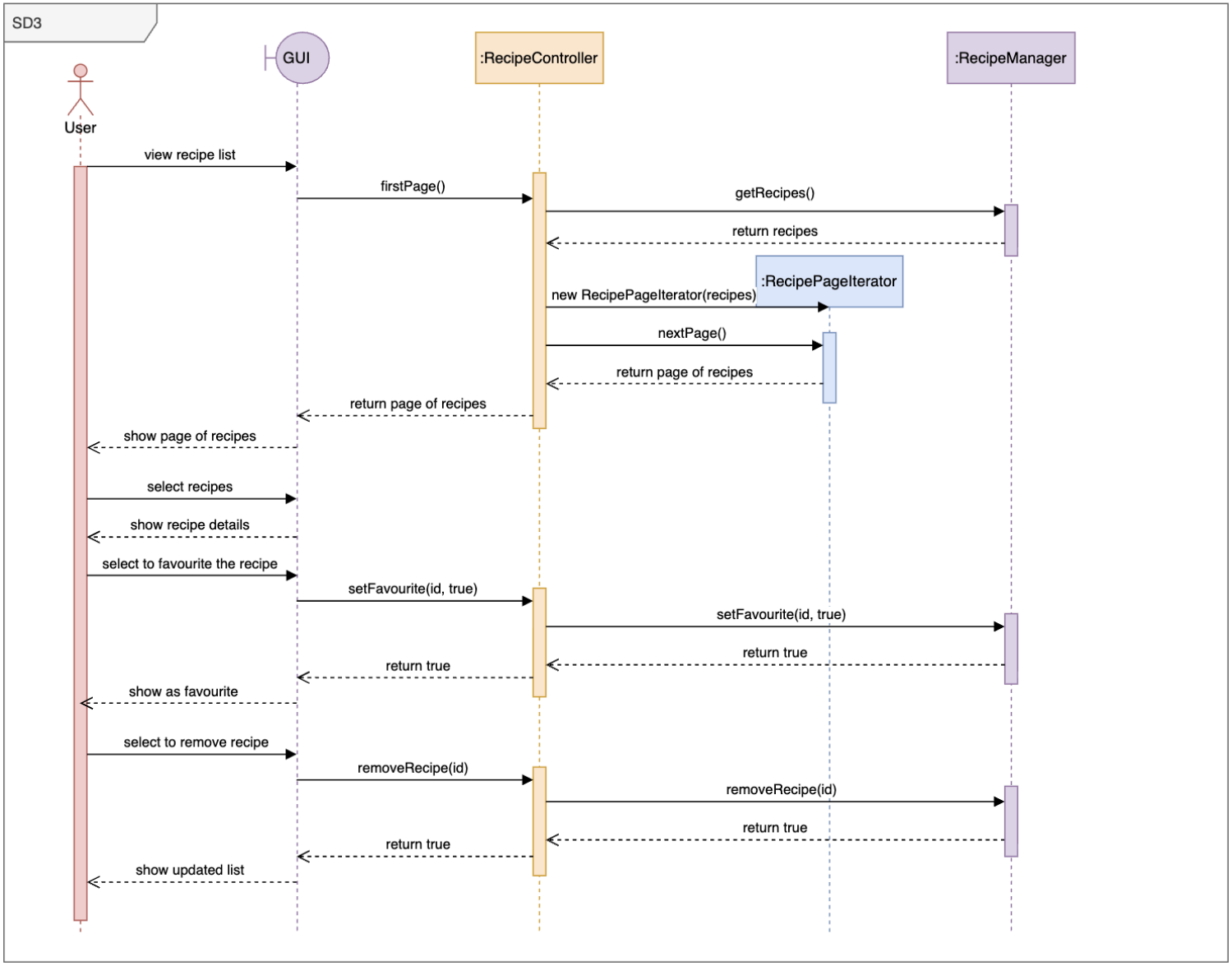
- **Postconditions:** The user's updated preferences and dietary restrictions are saved and available to relevant AI features
- **Related Feature(s):** F11 – User Preference Profiles and Dietary Restrictions

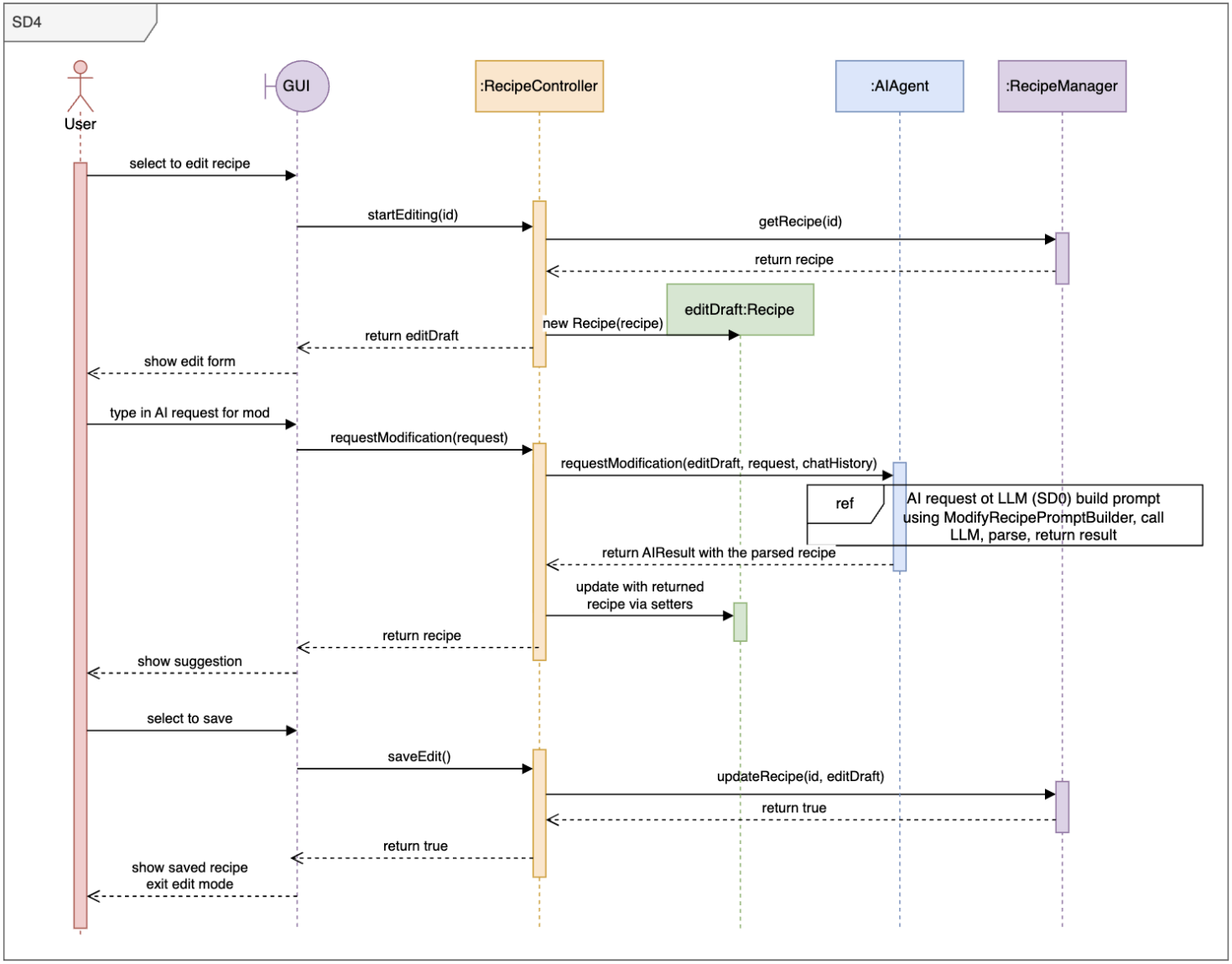
## 2.3 Sequence Diagrams

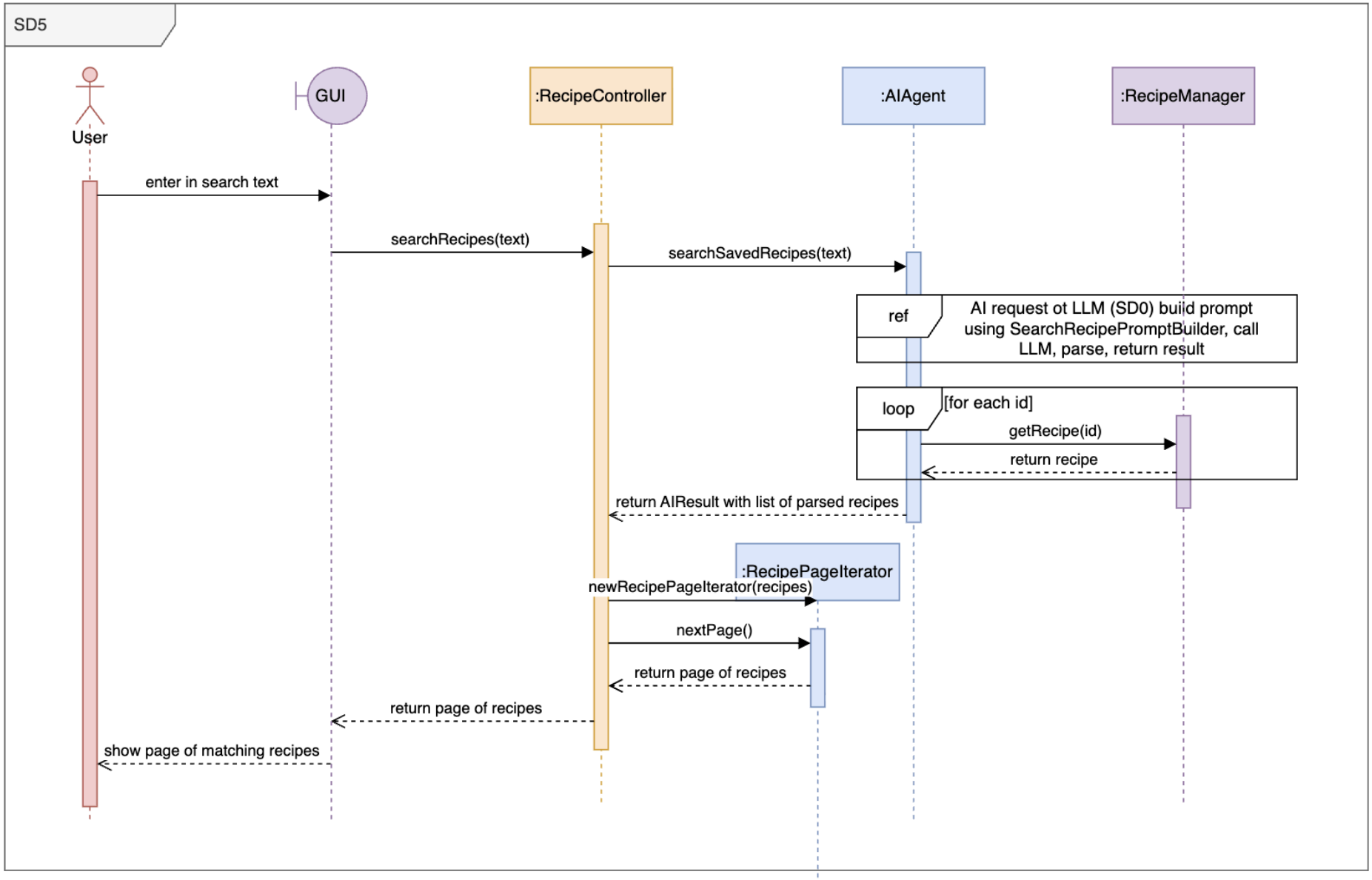
[Click here to view the full-size class diagram]

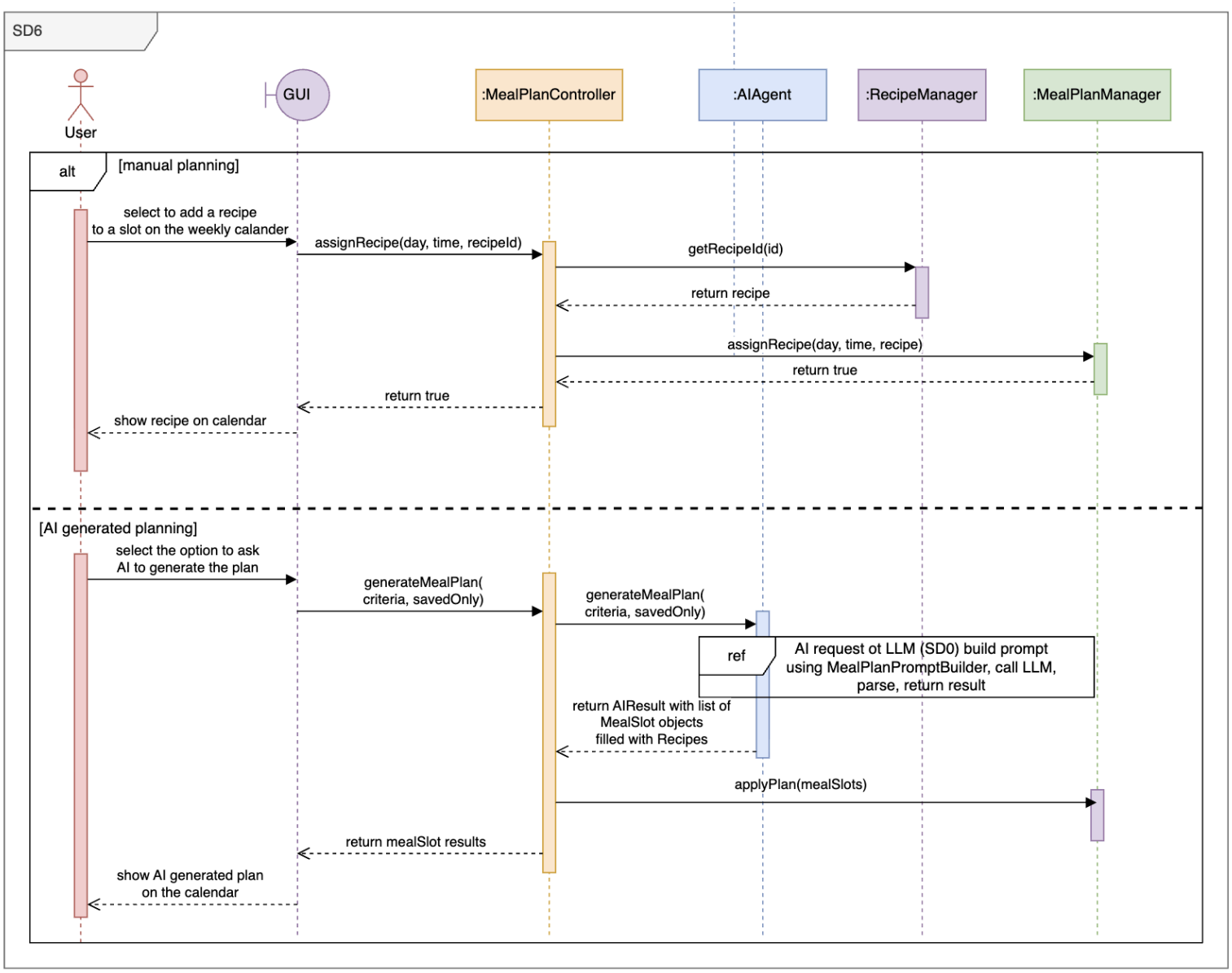


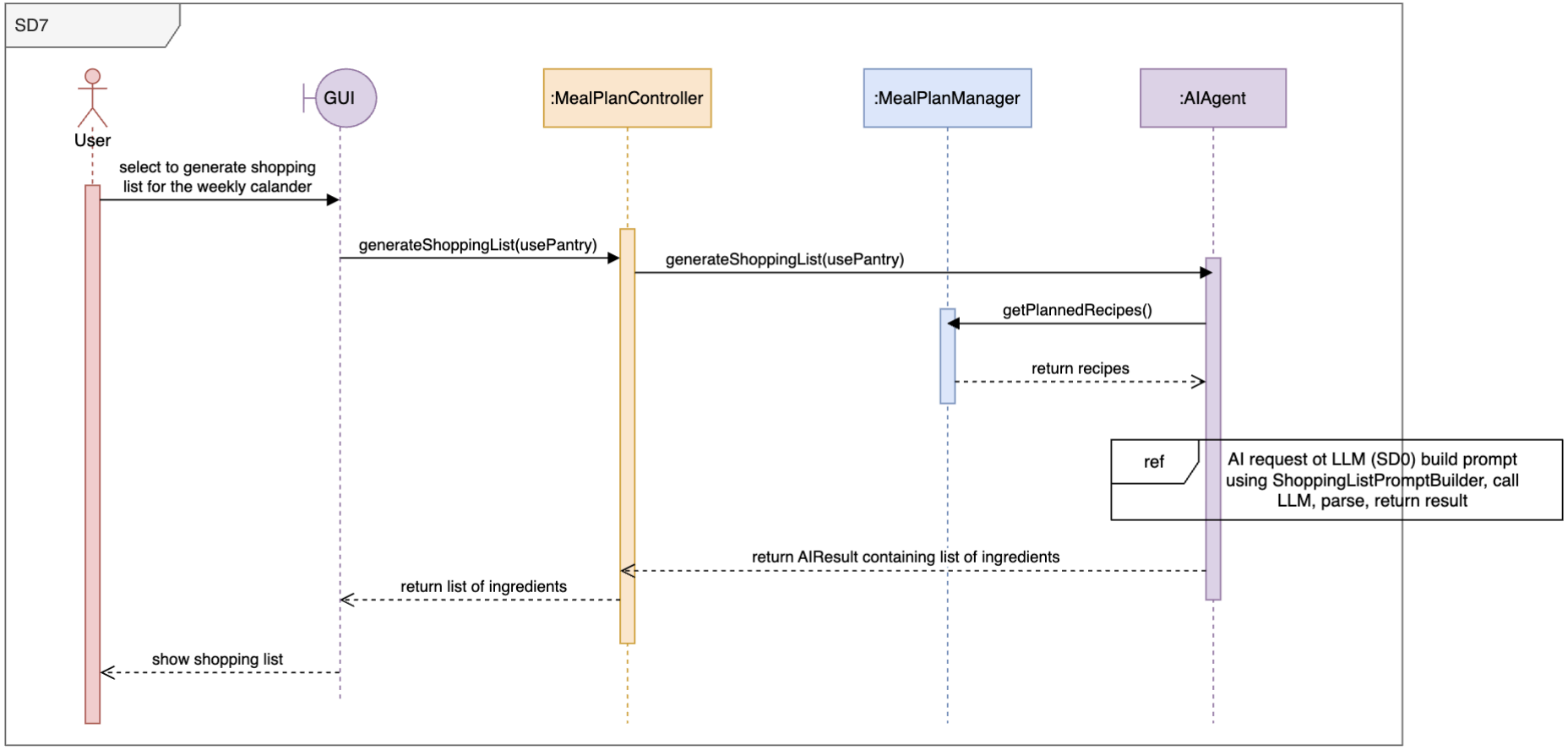

SD2
User
GUI
:RecipeController
:AIAgent
:RecipeManager
paste in recipe text
addPersonalRecipe(text)
parsePersonalRecipe(text)
ref
AI request ot LLM (SD0) build prompt using ParseRecipePromptBuilder, call LLM, parse, return result
return AIResult with the parsed recipe
saveRecipe(recipe)
return true
return recipe
show added recipe

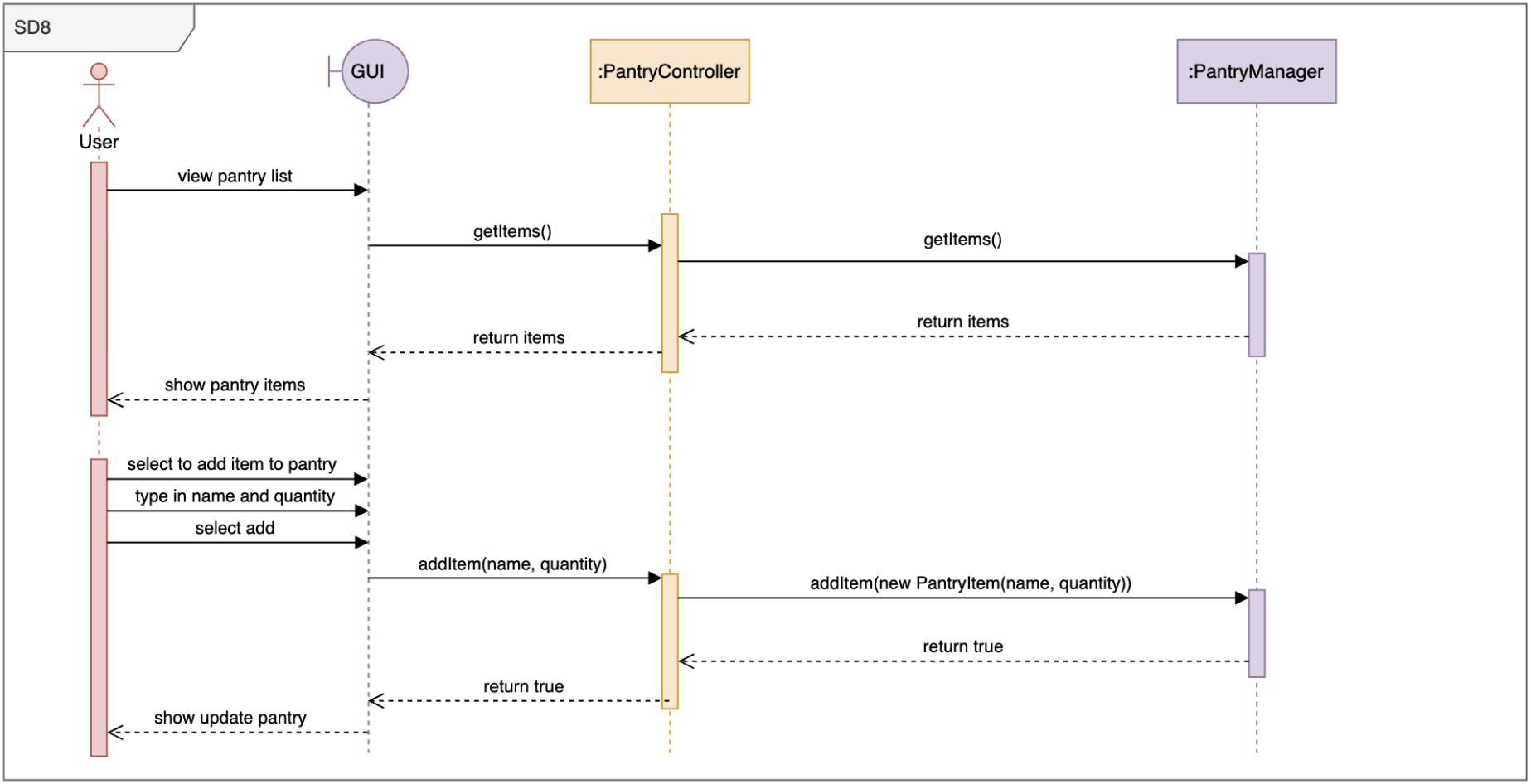

SD3
User
GUI
:RecipeController
:RecipeManager
:RecipePageIterator
view recipe list
firstPage()
getRecipes()
return recipes
new RecipePageIterator(recipes)
nextPage()
return page of recipes
return page of recipes
show page of recipes
select recipes
show recipe details
select to favourite the recipe
setFavourite(id, true)
setFavourite(id, true)
return true
return true
show as favourite
select to remove recipe
removeRecipe(id)
removeRecipe(id)
return true
return true
show updated list

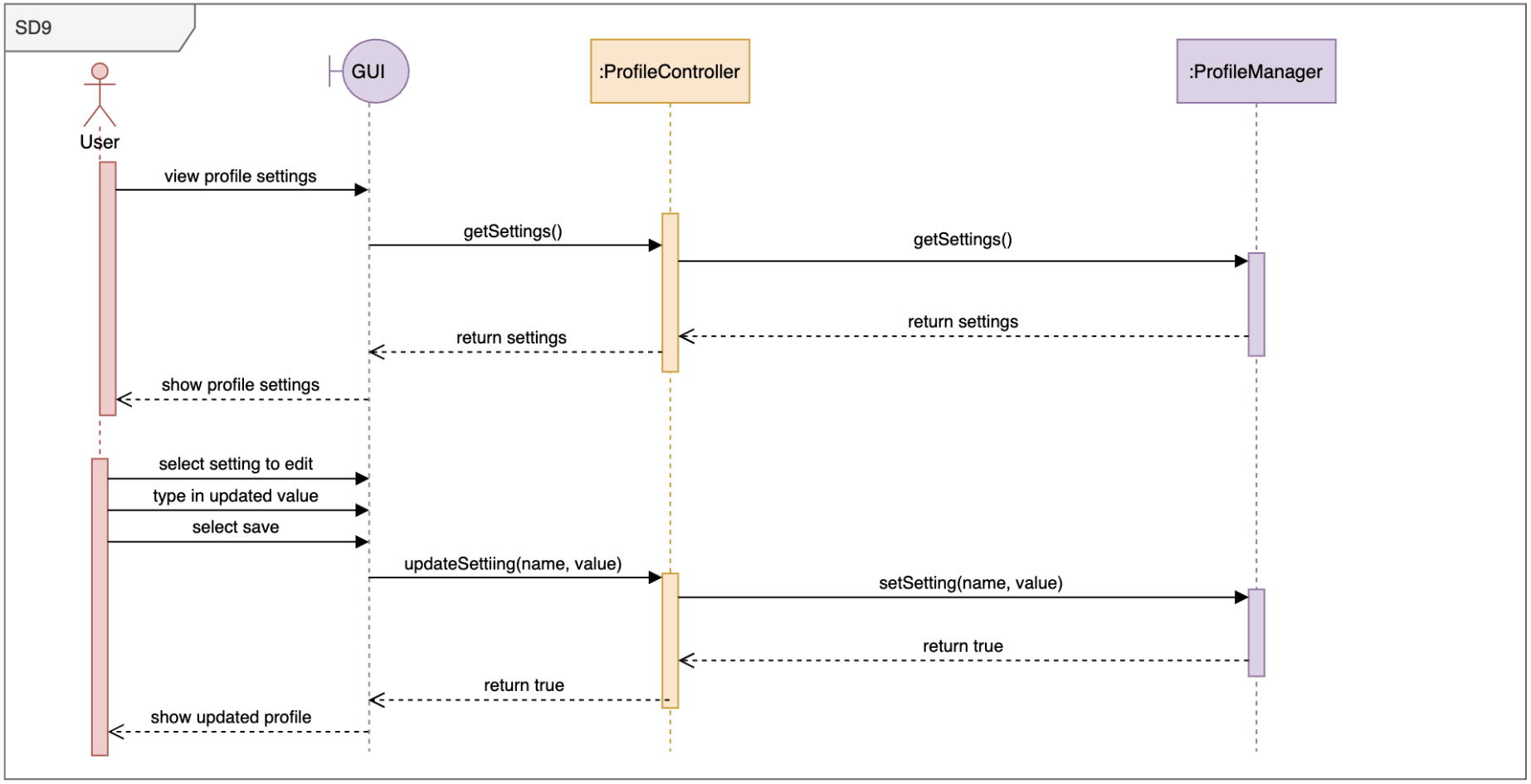

SD4
User
GUI
:RecipeController
:AIAgent
:RecipeManager
select to edit recipe
startEditing(id)
getRecipe(id)
return recipe
editDraft:Recipe
new Recipe(recipe)
return editDraft
show edit form
type in AI request for mod
requestModification(request)
requestModification(editDraft, request, chatHistory)
ref
AI request ot LLM (SD0) build prompt using ModifyRecipePromptBuilder, call LLM, parse, return result
return AIResult with the parsed recipe
update with returned recipe via setters
return recipe
show suggestion
select to save
saveEdit()
updateRecipe(id, editDraft)
return true
return true
show saved recipe
exit edit mode

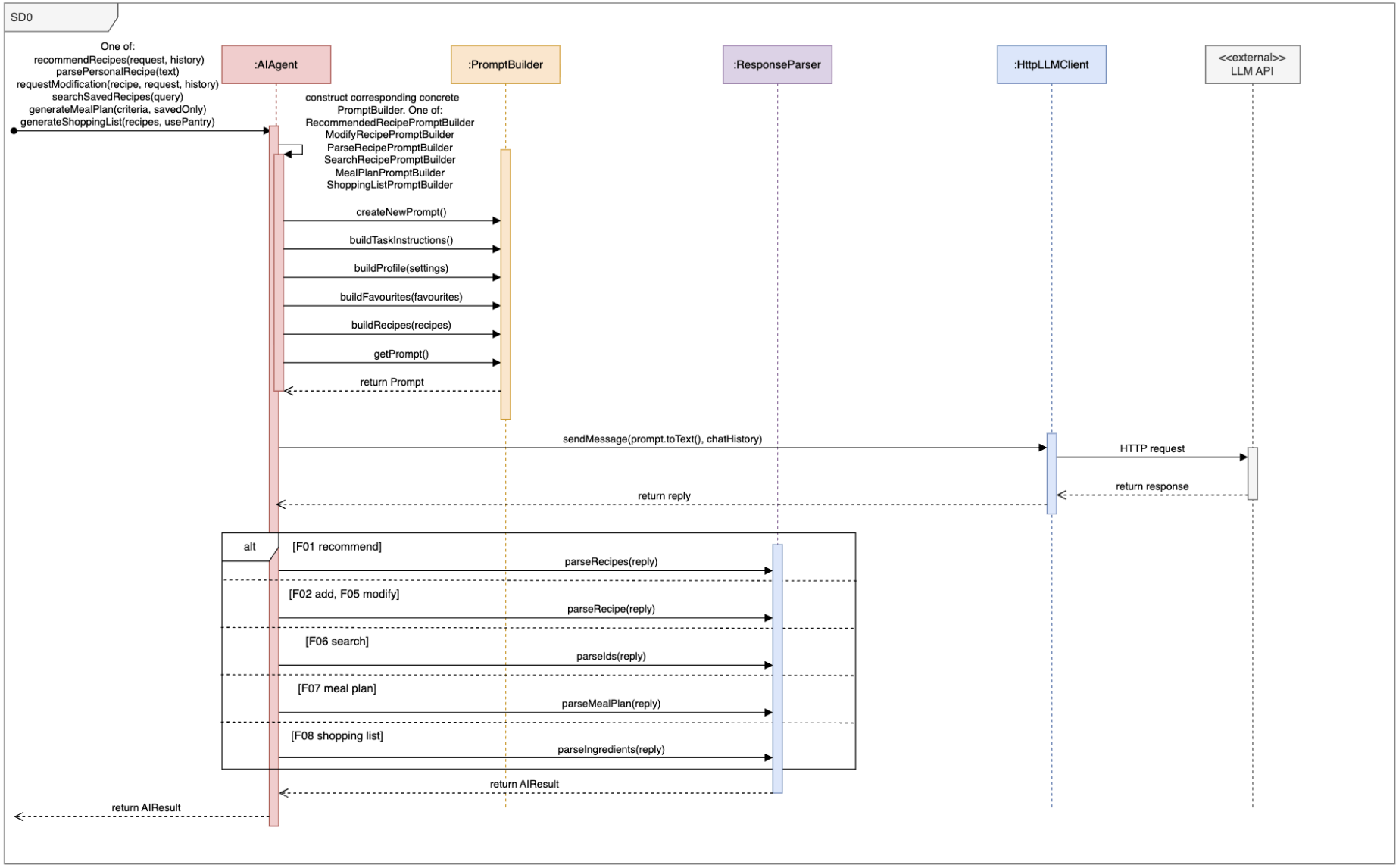

SD5
User
GUI
:RecipeController
:AIAgent
:RecipeManager
enter in search text
searchRecipes(text)
searchSavedRecipes(text)
ref
AI request ot LLM (SD0) build prompt using SearchRecipePromptBuilder, call LLM, parse, return result
loop
[for each id]
getRecipe(id)
return recipe
return AIResult with list of parsed recipes
:RecipePageIterator
newRecipePageIterator(recipes)
nextPage()
return page of recipes
return page of recipes
show page of matching recipes


SD6
User
GUI
:MealPlanController
:AIAgent
:RecipeManager
:MealPlanManager
alt
[manual planning]
select to add a recipe
to a slot on the weekly calander
assignRecipe(day, time, recipeId)
getRecipeId(id)
return recipe
assignRecipe(day, time, recipe)
return true
return true
show recipe on calendar
[AI generated planning]
select the option to ask
AI to generate the plan
generateMealPlan(
criteria, savedOnly)
generateMealPlan(
criteria, savedOnly)
ref
AI request ot LLM (SD0) build prompt
using MealPlanPromptBuilder, call LLM,
parse, return result
return AIResult with list of
MealSlot objects
filled with Recipes
applyPlan(mealSlots)
return mealSlot results
show AI generated plan
on the calendar


SD7
User
GUI
:MealPlanController
:MealPlanManager
:AIAgent
select to generate shopping list for the weekly calander
generateShoppingList(usePantry)
generateShoppingList(usePantry)
getPlannedRecipes()
return recipes
ref
AI request ot LLM (SD0) build prompt using ShoppingListPromptBuilder, call LLM, parse, return result
return AIResult containing list of ingredients
return list of ingredients
show shopping list


SD8
User
GUI
:PantryController
:PantryManager
view pantry list
getItems()
getItems()
return items
return items
show pantry items
select to add item to pantry
type in name and quantity
select add
addItem(name, quantity)
addItem(new PantryItem(name, quantity))
return true
return true
show update pantry


SD9
User
GUI
:ProfileController
:ProfileManager
view profile settings
getSettings()
getSettings()
return settings
return settings
show profile settings
select setting to edit
type in updated value
select save
updateSettiing(name, value)
setSetting(name, value)
return true
return true
show updated profile


SD0
One of:
recommendRecipes(request, history)
parsePersonalRecipe(text)
requestModification(recipe, request, history)
searchSavedRecipes(query)
generateMealPlan(criteria, savedOnly)
generateShoppingList(recipes, usePantry)
:AIAgent
:PromptBuilder
:ResponseParser
:HttpLLMClient
<<external>>
LLM API
construct corresponding concrete
PromptBuilder. One of:
RecommendedRecipePromptBuilder
ModifyRecipePromptBuilder
ParseRecipePromptBuilder
SearchRecipePromptBuilder
MealPlanPromptBuilder
ShoppingListPromptBuilder
createNewPrompt()
buildTaskInstructions()
buildProfile(settings)
buildFavourites(favourites)
buildRecipes(recipes)
getPrompt()
return Prompt
sendMessage(prompt.toText(), chatHistory)
HTTP request
return response
return reply
alt
[F01 recommend]
parseRecipes(reply)
[F02 add, F05 modify]
parseRecipe(reply)
[F06 search]
parseIds(reply)
[F07 meal plan]
parseMealPlan(reply)
[F08 shopping list]
parseIngredients(reply)
return AIResult
return AIResult

**Task 3: Feature-to-Design Traceability**

| Feature | Description | Type | Related Use Case | Classes | Key Methods | Sequence Diagram | Design Pattern(s) |
|---|---|---|---|---|---|---|---|
| F01 | Recipe Recommendation | AI-Based | UC01 | RecipeController, AIAgent, RecipeManager, Recipe, Ingredient, RecommendedRecipePromptBuilder, Prompt, HttpLLMClient, ChatMessage, ResponseParser | requestRecommendations(), recommendRecipes(), saveRecommendedRecipe()saveRecipe(), createNewPrompt(), buildTaskInstructions(), sendMessage(), parseRecipes() | SD1, SD0 | Facade, Builder, Adapter, Singleton |
| F02 | Add Personal Recipe To Recipe List | AI-Based | UC02 | RecipeController, AIAgent, RecipeManager, Recipe, Ingredient, ParseRecipePromptBuilder, Prompt, HttpLLMClient, ChatMessage, ResponseParser | addPersonalRecipe(), parsePersonalRecipe(), saveRecipe(), createNewPrompt(), buildTaskInstructions(), sendMessage(), parseRecipe() | SD2, SD0 | Facade, Builder, Adapter, Singleton |
| F03 | View Saved Recipes | Deterministic | UC03 | RecipeController, RecipeManager, Recipe, RecipePageIterator | firstPage(), getRecipes(), nextPage(), previousPage() | SD3 | Iterator, Singleton |
| F04 | Edit Saved Recipe | Deterministic | UC04 | RecipeController, RecipeManager, AIAgent, Recipe, Ingredient | startEditing(), getRecipe(), requestModification(), updateRecipe(), saveEdit() | SD4, SD0 | Singleton |
| F05 | Recipe Modification | AI-Based | UC05 | RecipeController, RecipeManager, Recipe, | startEditing(), getRecipe(), requestModification(), | SD4, SD0 | Facade, Builder, Adapter, Singleton |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | Help | | | Ingredient, ModifyRecipePromptBuilder, Prompt, HttpLLMClient, ChatMessage, ResponseParser | updateRecipe(), saveEdit(), createNewPrompt(), buildTaskInstructions(), sendMessage(), parseRecipe() | | |
| F06 | Search Saved Recipes | AI-Based | UC06 | RecipeController, RecipeManager, Recipe AIAgent, RecipePageIterator, SearchRecipePromptBuilder, Prompt, HttpLLMClient, ChatMessage, ResponseParser | searchRecipes(), searchSavedRecipes(), nextPage(), getRecipe(), createNewPrompt(), buildRecipes(), buildTaskInstructions(), sendMessage(), parseIds() | SD5, SD0 | Facade, Builder, Adapter, Singleton, Iterator |
| F07 | Weekly Meal Planning | Hybrid | UC07 | RecipeManager, AI Agent, MealPlanManager, MealPlanController, MealSlot, MealPlanPromptBuilder, Prompt, HttpLLMClient, ChatMessage, ResponseParser | assignRecipe(), getRecipeId(), generateMealPlan(), applyPlan(), createNewPrompt(), buildRecipes(), buildTaskInstructions(), sendMessage(), parseMealPlan() | SD6, SD0 | Facade, Builder, Adapter, Singleton |
| F08 | Shopping List Generation | AI-Based | UC08 | AI Agent, MealPlanManager, MealPlanController, PantryManager, PantryItem, ShoppingListPromptBuilder, Prompt, HttpLLMClient, ChatMessage, ResponseParser | generateShoppingList(), getPlannedRecipes(), createNewPrompt(), buildRecipes(), buildPantry(), buildTaskInstructions(), sendMessage(), | SD7, SD0 | Facade, Builder, Adapter, Singleton |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | parseIngredients() | | |
| F09 | Favourite Recipes | Deterministic | UC09 | RecipeController, RecipeManager, Recipe, RecipePageIterator | firstPage(), getRecipes(), nextPage(), setFavourite(), removeRecipe() | SD3 | Singleton, Iterator |
| F10 | Pantry Management | Deterministic | UC10 | PantryController, PantryManager, PantryItem | getItems(), addItem(), updateItem(), removeItem() | SD8 | Singleton |
| F11 | User Preference Profiles/Dietary Restrictions | Deterministic | UC11 | ProfileController, ProfileManager | getSettings(), updateSetting(), setSetting() | SD9 | Singleton |

**Task 4: Explain How Each Feature Is Realized**

**F01 - Recipe Recommendation**

**Use Case:** UC01 - Recommended Recipes

**Related Sequence Diagram:** SD1, SD0

**Classes involved:**

- RecipeController - coordinates the request, keeps the chat history, and saves a chosen recipe.
- AIAgent - entry point and coordinator for sending out the AI request
- RecipeManager - singleton manager of the app's saved list of recipes
- RecommendedRecipePromptBuilder - concrete builder of PromptBuilder, store code for building the system prompt for asking for a recipe recommendation and add the context sections to a Prompt.
- HttpLLMClient - adapter that communicates with the selected LLM
- ResponseParser - hold all the logic to convert the AI's various JSON replies into Recipe, MealSlots, and Ingredient objects.
- AIResult - wrapper for returning a parsed AI response along with error message if the request was flagged as unrelated or nonsense.

**Important methods:**

- RecipeController.requestRecommendations(request)
- AIAgent.recommendRecipes(request, chatHistory)
- RecommendRecipePromptBuilder.getPrompt()
- HttpLLMClient.sendMessage(systemPrompt, messages)
- ResponseParser.parseRecipes(json)
- RecipeController.saveRecommendedRecipe(recipe)
- RecipeManager.saveRecipe(recipe)

**Execution:**

When the user enters a request in the GUI, the GUI will call RecipeController.requestRecommendations(), which adds it to the chat history and calls AIAgent.recommendRecipes(). The agent reads the profile, pantry, and favourites from the managers and creates a RecommendRecipePromptBuilder and calls RecommendRecipePromptBuilder.getPrompt() which runs the standard build steps on a RecommendedRecipePromptBuilder to produce the prompt. The prompt and chat history are sent through HttpLLMClient.sendMessage() to the LLM, and the JSON reply is converted by ResponseParser.parseRecipes() into an AIResult holding Recipe objects. The result returns to the GUI, which shows the recipes, or shows an error message if the AI flagged the request. The user can keep chatting to refine the results. When they choose to save a recipe, saveRecommendedRecipe() calls RecipeManager.saveRecipe(), which checks if the name is unique, assigns an id, and stores it.

**F02 - Add Personal Recipe To Recipe List**

**Use Case: UC02 - Add Personal Recipe**

**Related Sequence Diagram: SD2, SD0**

**Classes involved:**

- RecipeController - coordinates the request, keeps the chat history, and saves a

chosen recipe.
- AIAgent - entry point and coordinator for sending out the AI request.
- ParseRecipePromptBuilder - concrete builder of PromptBuilder, stores code for building the system prompt for parsing pasted recipe text and adds the context sections to a Prompt.
- HttpLLMClient - adapter that communicates with the selected LLM.
- ResponseParser - holds all the logic to convert the AI's various JSON replies into Recipe, MealSlot, and Ingredient objects.
- AIResult - wrapper for returning a parsed AI response along with an error message if the request was flagged as unrelated or nonsense.
- RecipeManager - singleton manager of the app's saved list of recipes.
- Recipe - a recipe's name, ingredients, instructions, and other details, plus its favourite status.

**Important methods:**

- RecipeController.addPersonalRecipe(text)
- AIAgent.parsePersonalRecipe(text)
- ParseRecipePromptBuilder.getPrompt()
- HttpLLMClient.sendMessage(systemPrompt, messages)
- ResponseParser.parseRecipe(json)
- RecipeManager.saveRecipe(recipe)

**Execution:**

When the user pastes a recipe and submits it, pasteRecipe() passes the text to RecipeController.addPersonalRecipe(), which calls AIAgent.parsePersonalRecipe(). The agent creates a ParseRecipePromptBuilder and calls ParseRecipePromptBuilder.getPrompt(), producing a prompt that asks the AI to split the text into the standard recipe fields. The prompt and pasted text are sent through HttpLLMClient.sendMessage(), and ResponseParser.parseRecipe() converts the reply into an AIResult holding a Recipe, with any missing fields left blank. If parsing succeeded, the controller calls RecipeManager.saveRecipe(), which checks the name, assigns an id, and stores the recipe. The result returns to the GUI, which confirms the recipe was added, or shows a rejection message if the AI flagged the text as not a recipe or the name was already taken.

**F03 - View Saved Recipes**

**Use Case: UC03 - View Saved Recipes**

**Related Sequence Diagram: SD3**

**Classes involved:**

- RecipeController - coordinates the request, keeps the chat history, and saves a chosen recipe.
- RecipeManager - singleton manager of the app's saved list of recipes.
- RecipePageIterator - iterator that pages through a list of recipes a set number at a time.
- Recipe - a recipe's name, ingredients, instructions, and other details, plus its favourite status.

**Important methods:**

- RecipeController.firstPage()

- RecipeManager.getRecipes()
- RecipePageIterator.next()
- RecipeController.nextPage() / RecipeController.previousPage()
- RecipeController.removeRecipe(id)
- RecipeManager.removeRecipe(id)

**Execution:**

When the user opens their saved recipes, openRecipeList() calls RecipeController.firstPage(). The controller gets all saved recipes from RecipeManager.getRecipes(), creates a new RecipePageIterator over them, and calls next() to return the first page, which the GUI displays. When the user moves between pages, nextPage() and previousPage() ask the iterator for the next or previous page, and hasNextPage() and hasPreviousPage() tell the GUI which arrow buttons to enable. When the user selects a recipe, selectRecipe() shows its full details straight from the Recipe object already on the page. If the user removes it, removeRecipe() calls RecipeManager.removeRecipe(), then the GUI calls firstPage() again so the list is rebuilt without the removed recipe. If the recipe no longer exists, the manager returns false and the GUI shows an error and refreshes the list. If there are no saved recipes, the iterator returns an empty page and the list is shown empty.

**F04 - Edit Saved Recipes**

**Use Case:** UC04 - Edit Saved Recipe

**Related Sequence Diagram: SD4, SD0**

**Classes involved:**

- RecipeController - coordinates the request, keeps the chat history, and saves a chosen recipe.
- RecipeManager - singleton manager of the app's saved list of recipes.
- Recipe - a recipe's name, ingredients, instructions, and other details, plus its favourite status.
- Ingredient - one ingredient's name and amount, used in recipes and shopping lists.

**Important methods:**

- RecipeController.startEditing(id)
- RecipeManager.getRecipe(id)
- Recipe(other: Recipe) (copy constructor)
- RecipeController.saveEdit()
- RecipeManager.updateRecipe(id, updated)
- RecipeController.cancelEdit()

**Execution:**

When the user selects Edit on a recipe, clickEdit() calls RecipeController.startEditing(). The controller gets the saved recipe from RecipeManager.getRecipe() and makes an edit draft with the Recipe copy constructor, which keeps the id and deep-copies the ingredients, so the saved recipe is untouched while the user edits. The GUI shows the draft's fields, and the user's changes are written into the draft through its setters. When the user selects Save, saveEdit() calls RecipeManager.updateRecipe(), which blocks the save if the new name belongs to a different recipe and otherwise copies the draft's fields into the stored recipe. Blank

fields are allowed. The GUI then shows the updated recipe. If the user selects Cancel, cancelEdit() discards the draft and nothing is saved.

**F05 - Recipe Modification Help**

**Use Case: UC05 - Get Recipe Modification Help**

**Related Sequence Diagram: SD4, SD0**

**Classes involved:**

- RecipeController - coordinates the request, keeps the chat history, and saves a chosen recipe.
- AIAgent - entry point and coordinator for sending out the AI request.
- ProfileManager - singleton manager of the user's preference and dietary settings.
- PantryManager - singleton manager of the user's pantry items.
- RecipeManager - singleton manager of the app's saved list of recipes.
- ModifyRecipePromptBuilder - concrete builder of PromptBuilder, stores code for building the system prompt for asking for recipe modifications and adds the context sections to a Prompt.
- HttpLLMClient - adapter that communicates with the selected LLM.
- ResponseParser - holds all the logic to convert the AI's various JSON replies into Recipe, MealSlot, and Ingredient objects.
- AIResult - wrapper for returning a parsed AI response along with an error message if the request was flagged as unrelated or nonsense.
- ChatMessage - one turn of a conversation with the AI, labelled with its author.
- Recipe - a recipe's name, ingredients, instructions, and other details, plus its favourite status.
    - 

**Important methods:**

- RecipeController.requestModification(request)
- AIAgent.requestModification(recipe, request, chatHistory)
- ModifyRecipePromptBuilder.getPrompt()
- HttpLLMClient.sendMessage(systemPrompt, messages)
- ResponseParser.parseRecipe(json)
- RecipeController.saveEdit()

**Execution:**

While editing a recipe, the user describes the change they want in the AI chat. askAI() calls RecipeController.requestModification(), which adds the message to the chat history and calls AIAgent.requestModification() with the edit draft. The agent reads the profile and pantry from the managers and creates a ModifyRecipePromptBuilder and then calls ModifyRecipePromptBuilder.getPrompt(), passing the draft as the recipe context. The prompt and the full chat history are sent through HttpLLMClient.sendMessage(), and ResponseParser.parseRecipe() returns an AIResult whose message holds the AI's advice and, if the user asked for the change to be applied, a modified Recipe. The controller adds the AI's reply to the chat history and, when a recipe is returned, copies its fields into the draft while keeping the draft's id. The GUI shows the advice and the updated draft, and the user can keep chatting. The user then selects Save or Cancel, which works as in F04. If the request is unrelated to recipes, the AI flags it and the GUI shows an error reply instead.

**F06 - Search Saved Recipes**

**Use Case: UC06 - Search Saved Recipes**

**Related Sequence Diagram: SD5, SD0**

**Classes involved:**

- RecipeController - coordinates the request, keeps the chat history, and saves a chosen recipe.
- AIAgent - entry point and coordinator for sending out the AI request.
- RecipeManager - singleton manager of the app's saved list of recipes.
- SearchRecipePromptBuilder - concrete builder of PromptBuilder, stores code for building the system prompt for searching saved recipes and adds the context sections to a Prompt.
- HttpLLMClient - adapter that communicates with the selected LLM.
- ResponseParser - holds all the logic to convert the AI's various JSON replies into Recipe, MealSlot, and Ingredient objects.
- AIResult - wrapper for returning a parsed AI response along with an error message if the request was flagged as unrelated or nonsense.
- RecipePageIterator - iterator that pages through a list of recipes a set number at a time.
- Recipe - a recipe's name, ingredients, instructions, and other details, plus its favourite status.

**Important methods:**

- RecipeController.searchRecipes(query)
- AIAgent.searchSavedRecipes(query)
- SearchRecipePromptBuilder.getPrompt()
- HttpLLMClient.sendMessage(systemPrompt, messages)
- ResponseParser.parseIds(json)
- RecipeManager.getRecipe(id)
- RecipePageIterator.next()

**Execution:**

When the user enters a search description on the recipe list, enterSearch() calls RecipeController.searchRecipes(), which calls AIAgent.searchSavedRecipes(). The agent gets all saved recipes from RecipeManager.getRecipes() and creates a SearchRecipePromptBuilder.getPrompt() and then calls SearchRecipePromptBuilder.getPrompt() which calls the buildRecipes prompt builder passing the recipes with their ids as context. The prompt and query are sent through HttpLLMClient.sendMessage(), and ResponseParser.parseIds() extracts the ids of the matching recipes. The agent looks up each id with RecipeManager.getRecipe(), so the AI can only return recipes that really exist, and wraps the matches in an AIResult. The controller creates a new RecipePageIterator over the matches and returns the first page, which the GUI displays. The user pages through the results as in F03, and clearSearch() returns to the full list. If nothing matches, the list is empty, and if the query is unrelated to recipes, the GUI shows an error message.

**F07 - Weekly Meal Planning**

**Use Case: UC07 - Plan Weekly Meals**

**Related Sequence Diagram: SD6, SD0**

**Classes involved:**

- MealPlanController - coordinates meal planning requests, assigns recipes to slots, and applies generated plans.
- MealPlanManager - singleton manager of the weekly meal plan's slots.
- MealSlot - one day and meal time in the weekly plan, holding at most one recipe.
- RecipeManager - singleton manager of the app's saved list of recipes.
- AIAgent - entry point and coordinator for sending out the AI request.
- ProfileManager - singleton manager of the user's preference and dietary settings.
- PantryManager - singleton manager of the user's pantry items.
- MealPlanPromptBuilder - concrete builder of PromptBuilder, stores code for building the system prompt for generating a weekly meal plan and adds the context sections to a Prompt.
- HttpLLMClient - adapter that communicates with the selected LLM.
- ResponseParser - holds all the logic to convert the AI's various JSON replies into Recipe, MealSlot, and Ingredient objects.
- AIResult - wrapper for returning a parsed AI response along with an error message if the request was flagged as unrelated or nonsense.
- Recipe - a recipe's name, ingredients, instructions, and other details, plus its favourite status.

**Important methods:**

- MealPlanController.getSlots()
- MealPlanController.assignRecipe(day, time, recipeId)
- MealPlanManager.assignRecipe(day, time, recipe)
- MealPlanController.clearSlot(day, time)
- AIAgent.generateMealPlan(criteria, savedOnly)
- MealPlanPromptBuilder.getPrompt()
- ResponseParser.parseMealPlan(json, savedRecipes)
- MealPlanManager.applyPlan(generated)

**Execution:**

When the user opens the planner, MealPlanController.getSlots() returns the week's slots from MealPlanManager, which the GUI shows as a calendar. To plan manually, the user picks a day and meal time, chooses a saved recipe using the F03 list or F06 search, and addToSlot() calls MealPlanController.assignRecipe(). The controller gets the recipe from RecipeManager.getRecipe() and passes it to MealPlanManager.assignRecipe(), which checks the meal time and fills the slot. Recipes can be replaced the same way or removed with clearSlot(). To generate a plan, the user describes their criteria and chooses whether to use only saved recipes, and generatePlan() calls MealPlanController.generateMealPlan(), which calls AIAgent.generateMealPlan(). The agent reads the profile, pantry, favourites, and all saved recipes, and creates a MealPlanPromptBuilder and calls MealPlanPromptBuilder.getPrompt() which calls the buildFavourites, buildRecipes, buildPantry, buildProfile prompt builders with the data. The prompt is sent through HttpLLMClient.sendMessage(), and ResponseParser.parseMealPlan() turns the reply into MealSlot objects, matching returned ids to saved recipes and dropping any unknown ids so those slots stay empty. If the result succeeded, the controller calls MealPlanManager.applyPlan(), and the GUI redraws the completed calendar. If the AI

can't build a suitable plan, the GUI shows a message asking for more information or suggesting relaxed requirements.

**F08 - Shopping List Generation**

**Use Case: UC08 - Generate Shopping List**

**Related Sequence Diagram: SD7, SD0**

**Classes involved:**

- MealPlanController - coordinates meal planning requests, assigns recipes to slots, and applies generated plans.
- AIAgent - entry point and coordinator for sending out the AI request.
- MealPlanManager - singleton manager of the weekly meal plan's slots.
- PantryManager - singleton manager of the user's pantry items.
- ShoppingListPromptBuilder - concrete builder of PromptBuilder, stores code for building the system prompt for generating a shopping list and adds the context sections to a Prompt.
- HttpLLMClient - adapter that communicates with the selected LLM.
- ResponseParser - holds all the logic to convert the AI's various JSON replies into Recipe, MealSlot, and Ingredient objects.
- AIResult - wrapper for returning a parsed AI response along with an error message if the request was flagged as unrelated or nonsense.
- Ingredient - one ingredient's name and amount, used in recipes **and shopping lists.**

**Important methods:**

- MealPlanController.generateShoppingList(usePantry)
- AIAgent.generateShoppingList(usePantry)
- MealPlanManager.getPlannedRecipes()
- PantryManager.getItems()
- ShoppingListPromptBuilder.getPrompt()
- HttpLLMClient.sendMessage(systemPrompt, messages)
- ResponseParser.parseIngredients(json)

**Execution:**

When the user selects Generate Shopping List on the meal planner and chooses whether to factor in their pantry, clickShoppingList() calls MealPlanController.generateShoppingList(), which calls AIAgent.generateShoppingList(). The agent gets the planned recipes from MealPlanManager.getPlannedRecipes() and, only if the pantry is to be used, the pantry items from PantryManager.getItems(). It creates a ShoppingListPromptBuilder and calls ShoppingListPromptBuilder.getPrompt(), which only calls buildPantry to add it to the system prompt if it’s used. The prompt is sent through HttpLLMClient.sendMessage(), and ResponseParser.parseIngredients() converts the reply into a list of Ingredient objects holding what still needs to be bought and how much. The result returns to the GUI, which shows the list. If the user already has everything, the list is empty and the GUI shows that no items are needed, and if a recipe is missing ingredient information, the result's message tells the user.

**F09 - Favourite Recipes**

**Use Case: UC09 - Favourite Saved Recipe**

**Related Sequence Diagram: SD3**

**Classes involved:**

- RecipeController - coordinates the request, keeps the chat history, and saves a chosen recipe.
- RecipeManager - singleton manager of the app's saved list of recipes.
- Recipe - a recipe's name, ingredients, instructions, and other details, plus its favourite status.
- PromptBuilder - abstract builder that holds the shared base rules and build steps for every AI prompt.

**Important methods:**

- Recipe.isFavourite()
- RecipeController.setFavourite(id, fav)
- RecipeManager.setFavourite(id, fav)
- RecipeManager.getFavourites()
- PromptBuilder.buildFavourites(recipes)

**Execution:**

When the user views the recipe list or a recipe's details, the GUI reads Recipe.isFavourite() to show the favourite indicator. When the user selects Favourite or Unfavourite, clickFavourite() calls RecipeController.setFavourite(), which calls RecipeManager.setFavourite(). The manager only accepts recipes in its saved list, so unsaved recommendations can never be favourited, and it updates the recipe's favourite flag. The GUI then shows the updated indicator. Whenever an AI feature that uses favourites runs (F01 and F07), AIAgent reads them with RecipeManager.getFavourites() and passes them to PromptBuilder.buildFavourites(), which adds them to the prompt.

**F10 - Pantry Management**

**Use Case: UC10 - Manage Pantry List**

**Related Sequence Diagram: SD8**

**Classes involved:**

- PantryController - coordinates pantry requests and passes changes to the PantryManager.
- PantryManager - singleton manager of the user's pantry items.
- PantryItem - one pantry ingredient's name and quantity.

**Important methods:**

- PantryController.getItems()
- PantryController.addItem(name, quantity)
- PantryManager.addItem(item)
- PantryController.updateItem(oldName, name, quantity)
- PantryManager.updateItem(name, updated)
- PantryController.removeItem(name)

**Execution:**

When the user opens the pantry, openPantry() calls PantryController.getItems(), which returns the items from PantryManager, and the GUI lists each item's name and

quantity. To add an item, the user enters a name and quantity, and PantryController.addItem() creates a PantryItem and passes it to PantryManager.addItem(). The manager rejects blank fields, names over the character limit, and names that already exist, ignoring differences in case. Editing works the same way through updateItem(), which also moves the item to its new name if it was renamed, and removeItem() deletes an item. After each change, the GUI shows the updated list, or an error message if the change was rejected. The AI features read the same items through PantryManager.getItems().

**F11 - User Preference Profiles and Dietary Restrictions**

**Use Case: UC11 - Manage Preference Profile**

**Related Sequence Diagram: SD9**

**Classes involved:**

- ProfileController - coordinates profile requests and passes changes to the ProfileManager.
- ProfileManager - singleton manager of the user's preference and dietary settings.
- PromptBuilder - abstract builder that holds the shared base rules and build steps for every AI prompt.

**Important methods:**

- ProfileController.getSettings()
- ProfileController.updateSetting(name, value)
- ProfileManager.setSetting(name, value)
- PromptBuilder.buildProfile(profile)

**Execution:**

When the user opens their profile, openProfile() calls ProfileController.getSettings(), which returns every setting from ProfileManager: dietary restrictions, preferred foods, disliked ingredients, nutrition goals, and other preferences. When the user edits a field and saves, editField() calls ProfileController.updateSetting(), which calls ProfileManager.setSetting(). The manager only accepts the fixed setting names and allows blank values. The GUI then shows the saved profile. Whenever an AI feature that uses the profile runs (F01, F05, and F07), AIAgent reads the settings and passes them to PromptBuilder.buildProfile(), which skips blank settings, so the AI ignores any preference the user left empty.